# UNIT - 3
# TRANSACTIONS

Transaction Concepts – ACID Properties – Schedule and immediate update – Shadow paging – ARIES Algorithm– Serializability – Transaction support in SQL – Need for Concurrency – Concurrency control –Two Phase Locking- Timestamp – Multiversion – Multiple Granularity locking – Deadlock Handling – Recovery Concepts – Recovery based on deferred.

## 3.1 Transaction Concepts:

In **Database Management Systems (DBMS)**, a transaction is a fundamental concept representing a set of logically related operations executed as a **single unit**. **Transactions** are essential for handling user requests to access and modify database contents, ensuring the database remains consistent and reliable despite various operations and potential interruptions.

Operations of Transaction

A user can make different types of requests to access and modify the contents of a database. So, we have different types of operations relating to a transaction. They are discussed as follows:

**i) Read(X)**

- A **read operation** is used to read the value of **X** from the database and store it in a buffer in the main memory for further actions such as displaying that value.
- Such an operation is performed when a user wishes just to see any content of the database and not make any changes to it. For example, when a user wants to check **his/her** account's balance, a read operation would be performed on user's account balance from the database.

**ii) Write(X)**

- A write operation is used to write the value to the database from the buffer in the main memory. For a write operation to be performed, first a read operation is performed to bring its value in buffer, and then some changes are made to it, **e.g.** some set of arithmetic operations are performed on it according to the user's request, then to store the modified value back in the database, a write operation is performed.
- **For example**, when a user requests to withdraw some money from his account, his account balance is fetched from the database using a read operation, then the amount to be deducted from the account is subtracted from this value, and then the obtained value is stored back in the database using a write operation.

**iii) Commit**

- This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction.
- To ensure that further operations of any other transaction are performed only after work of the current transaction is done, a commit operation is performed to the changes made by a transaction permanently to the database.

**iv) Roll Back**

- This operation is performed to bring the database to the last saved state when any transaction is interrupted in between due to any power, hardware, or software failure.
- In simple words, it can be said that a rollback operation does undo the operations of transactions that were performed before its interruption to achieve a safe state of the database and avoid any kind of ambiguity or inconsistency.

**Transaction Schedules:**
When multiple transaction requests are made at the same time, we need to decide their order of execution. Thus, a transaction schedule can be defined as a chronological order of execution of multiple transactions.

There are broadly two types of transaction schedules discussed as follows:

**i) Serial Schedule**

- In this kind of schedule, when multiple transactions are to be executed, they are executed serially, i.e. at one time only one transaction is executed while others wait for the execution of the current transaction to be completed. This ensures consistency in the database as transactions do not execute simultaneously.
- But, it increases the waiting time of the transactions in the queue, which in turn lowers the throughput of the system, i.e. number of transactions executed per time.
- To improve the throughput of the system, another kind of schedule are used which has some more strict rules which help the database to remain consistent even when transactions execute simultaneously.

**ii) Non-Serial Schedule**

- To reduce the waiting time of transactions in the waiting queue and improve the system efficiency, we use non-serial schedules which allow multiple transactions to start before a transaction is completely executed. This may sometimes result in inconsistency and errors in database operation.
- So, these errors are handled with specific algorithms to maintain the consistency of the database and improve **CPU**throughput as well.
- Non-serial schedules are also sometimes referred to as parallel schedules, as transactions execute in parallel in these kinds of schedules.

**Serializable**

- Serializability in DBMS is the property of a nonserial schedule that determines whether it would maintain the database consistency or not.
- The nonserial schedule which ensures that the database would be consistent after the transactions are executed in the order determined by that schedule is said to be Serializable Schedules.

The serial schedules always maintain database consistency as a transaction starts only when the execution of the other transaction has been completed under it.

- Thus, serial schedules are always serializable.
- A transaction is a series of operations, so various states occur in its completion journey.

They are discussed as follows:

**i) Active**

- It is the first stage of any transaction when it has begun to execute. The execution of the transaction takes place in this state.
- Operations such as insertion, deletion, or updation are performed during this state.
- During this state, the data records are under manipulation and they are not saved to the database, rather they remain somewhere in a buffer in the main memory.

**ii) Partially Committed**

- This state of transaction is achieved when it has completed most of the operations and is executing its final operation.

- It can be a signal to the commit operation, as after the final operation of the transaction completes its execution, the data has to be saved to the database through the commit operation.
- If some kind of error occurs during this state, the transaction goes into a failed state, else it goes into the Committed state.

**iii) Committed**

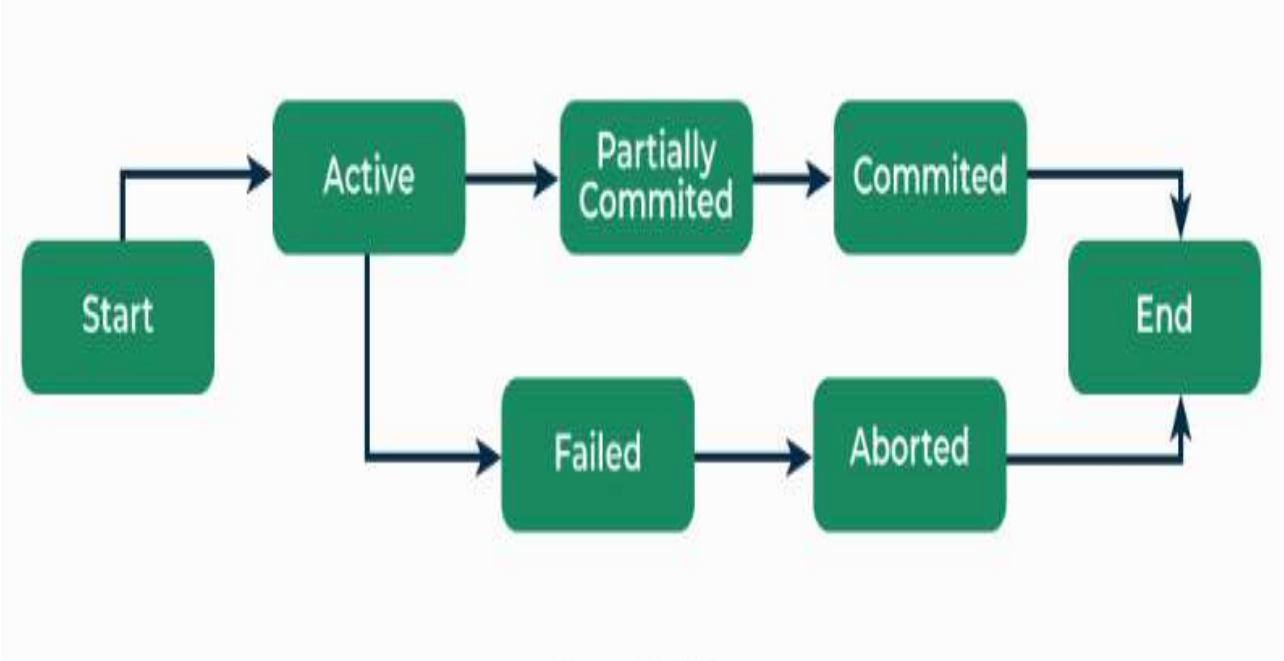
This state of transaction is achieved when all the transaction-related operations have been executed successfully along with the Commit operation, i.e. data is saved into the database after the required manipulations in this state. This marks the successful completion of a transaction.

**iv) Failed**

- If any of the transaction-related operations cause an error during the active or partially committed state, further execution of the transaction is stopped and it is brought into a failed state. Here, the database recovery system makes sure that the database is in a consistent state.

**v) Aborted**

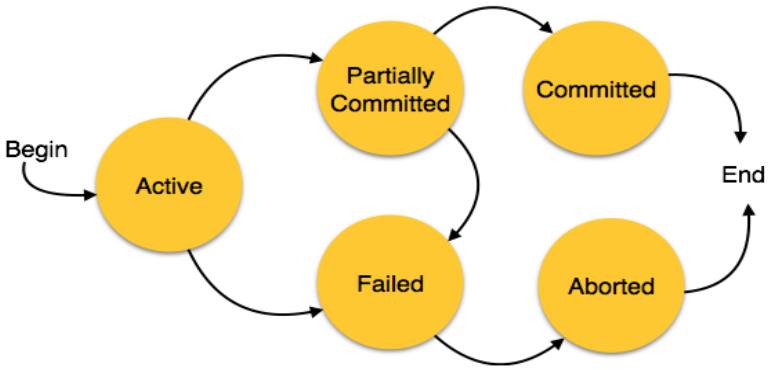
If the error is not resolved in the failed state, then the transaction is aborted and a rollback operation is performed to bring database to the the last saved consistent state. When the transaction is aborted, the database recovery module either restarts the transaction or kills it. The illustration below shows the various states that a transaction may encounter in its completion journey.




*Transaction in DBMS*

## States of Transactions

A transaction in a database can be in one of the following states –




- **Active** – In this state, the transaction is being executed. This is the initial state of every transaction.
- **Partially Committed** – When a transaction executes its final operation, it is said to be in a partially committed state.
- **Failed** – A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.
- **Aborted** – If any of the checks fails and the transaction has reached a failed state, then the recovery manager rolls back all its write operations on the database to bring the database back to its original state where it was prior to the execution of the transaction. Transactions in this state are called aborted. The database recovery module can select one of the two operations after a transaction aborts –
  - Re-start the transaction
  - Kill the transaction
- **Committed** – If a transaction executes all its operations successfully, it is said to be committed. All its effects are now permanently established on the database system.

## 3.2 ACID Properties:

**Properties of Transaction**

- As transactions deal with accessing and modifying the contents of the database, they must have some basic properties which help maintain the consistency and integrity of the database before and after the transaction. Transactions follow 4 properties, namely, **Atomicity, Consistency, Isolation, and Durability**.
- Generally, these are referred to as **ACID** properties of transactions in DBMS. ACID is the acronym used for transaction properties. A brief description of each property of the transaction is as follows.

**i) Atomicity**

- This property ensures that either all operations of a transaction are executed or it is aborted. In any case, a transaction can never be completed partially.
- Each transaction is treated as a single unit (like an atom). Atomicity is achieved through commit and rollback operations, i.e. changes are made to the database only if all operations related to a transaction are completed, and if it gets interrupted, any changes made are rolled back using rollback operation to bring the database to its last saved state.

**ii) Consistency**

- This property of a transaction keeps the database consistent before and after a transaction is completed.
- Execution of any transaction must ensure that after its execution, the database is either in its prior stable state or a new stable state.
- In other words, the result of a transaction should be the transformation of a database from one consistent state to another consistent state.
- Consistency, here means, that the changes made in the database are a result of logical operations only which the user desired to perform and there is not any ambiguity.

**iii) Isolation**

- This property states that two transactions must not interfere with each other, i.e. if some data is used by a transaction for its execution, then any other transaction can not concurrently access that data until the first transaction has completed.
- It ensures that the integrity of the database is maintained and we don't get any ambiguous values. Thus, any two transactions are isolated from each other.
- This property is enforced by the concurrency control subsystem of DBMS.

**iv) Durability**

- This property ensures that the changes made to the database after a transaction is completely executed, are durable.
- It indicates that permanent changes are made by the successful execution of a transaction.
- In the event of any system failures or crashes, the consistent state achieved after the completion of a transaction remains intact. The recovery subsystem of DBMS is responsible for enforcing this property.

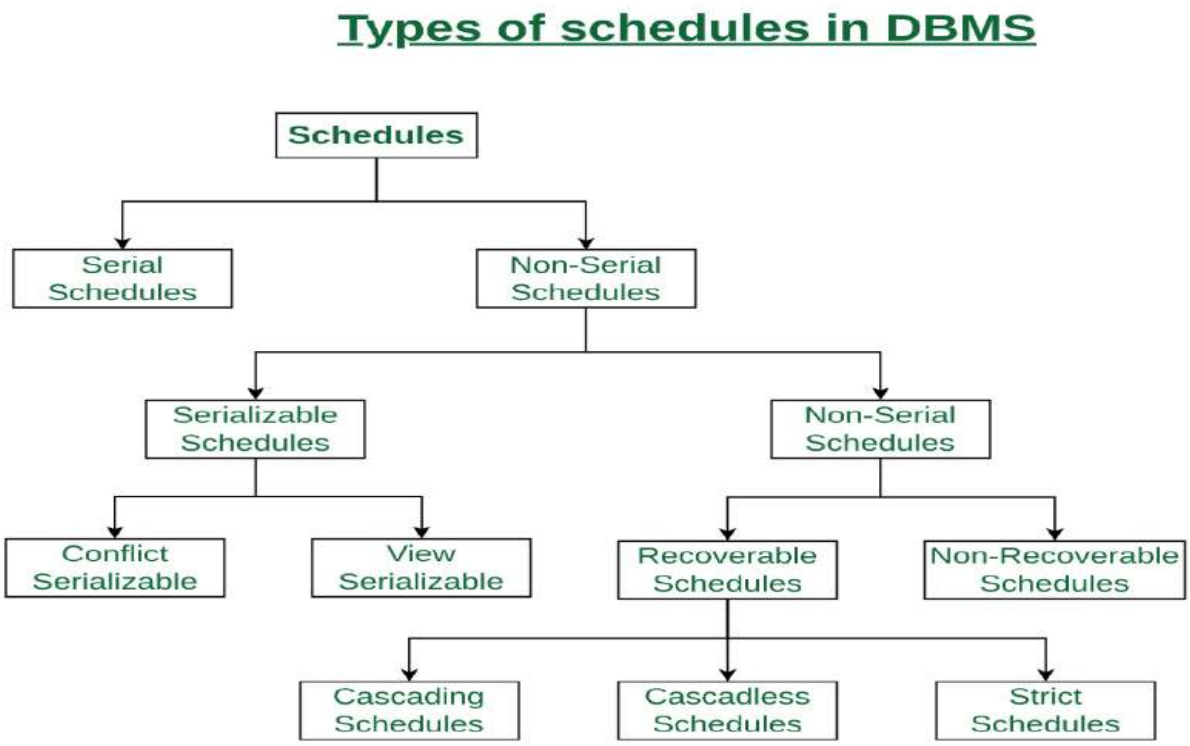
## ACID Properties

A transaction is a very small unit of a program and it may contain several lowlevel tasks. A transaction in a database system must maintain **A**tomicity, **C**onsistency, **I**solation, and **D**urability – commonly known as ACID properties – in order to ensure accuracy, completeness, and data integrity.

- **Atomicity** – This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none. There must be no state in a database where a transaction is left partially completed. States should be defined either before the execution of the transaction or after the execution/abortion/failure of the transaction.
- **Consistency** – The database must remain in a consistent state after any transaction. No transaction should have any adverse effect on the data residing in the database. If the database was in a consistent state before the execution of a transaction, it must remain consistent after the execution of the transaction as well.
- **Durability** – The database should be durable enough to hold all its latest updates even if the system fails or restarts. If a transaction updates a chunk of data in a database and commits, then the database will hold the modified data. If a transaction commits but the system fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.
- **Isolation** – In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. No transaction will affect the existence of any other transaction.

## 3.3 Schedule & Immediate Update:

Schedule is a process of lining the transactions and executing them one by one. When there are multiple transactions that are running in a concurrent manner and the order of operation is needed to be set so that the operations do not overlap each other, Scheduling is brought into play and the transactions are timed accordingly.




1. **Serial Schedules:** Schedules in which the transactions are executed non-interleaved, i.e., a serial schedule is one in which no transaction starts until a running transaction has ended are called serial schedules. **Example:** Consider the following schedule involving two transactions $T_1$ and $T_2$.

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| R(B) | |
| | W(B) |
| | R(A) |
| | R(B) |

where R(A) denotes that a read operation is performed on some data item 'A' This is a serial schedule since the transactions perform serially in the order $T_1$ —> $T_2$

2. **Non-Serial Schedule:** This is a type of Scheduling where the operations of multiple transactions are interleaved. This might lead to a rise in the concurrency problem. The transactions are executed in a non-serial manner, keeping the end result correct and same as the serial schedule. Unlike the serial schedule where one transaction must wait for another to complete all its operation, in the non-serial schedule, the other transaction proceeds without waiting for the previous transaction to complete. This sort of schedule does not provide any benefit of the concurrent transaction. It can be of two types namely, Serializable and Non-Serializable Schedule. The Non-Serial Schedule can be divided further into Serializable and Non-Serializable.
    1. **Serializable:** This is used to maintain the consistency of the database. It is mainly used in the Non-Serial scheduling to verify whether the scheduling will lead to any inconsistency or not. On the other hand, a serial schedule does not need the serializability because it follows a transaction only when the previous transaction is complete. The non-serial schedule is said to be in a serializable schedule only when it is equivalent to the serial schedules, for an n number of transactions. Since concurrency is allowed in this case thus, multiple transactions can execute concurrently. A serializable schedule helps in improving both resource utilization and CPU throughput. These are of two types:
        1. Conflict Serializable: A schedule is called conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations. Two operations are said to be conflicting if all conditions satisfy:
            - They belong to different transactions
            - They operate on the same data item
            - At Least one of them is a write operation

        2. View Serializable: A Schedule is called view serializable if it is view equal to a serial schedule (no overlapping transactions). A conflict schedule is a view serializable but if the serializability contains blind writes, then the view serializable does not conflict serializable.

    2. **Non-Serializable:** The non-serializable schedule is divided into two types, Recoverable and Non-recoverable Schedule.
        1. Recoverable Schedule: Schedules in which transactions commit only after all transactions whose changes they read commit are called recoverable schedules. In other words, if some transaction $T_j$ is reading value updated or written by some other transaction $T_i$, then the commit of $T_j$ must occur after the commit of $T_i$. **Example** – Consider the following schedule involving two transactions $T_1$ and $T_2$.

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| | W(A) |
| | R(A) |
| commit | |
| | commit |

This is a recoverable schedule since $T_1$ commits before $T_2$, that makes the value read by $T_2$ correct. There can be three types of recoverable schedule:

1. **Cascading Schedule:** Also called Avoids cascading aborts/rollbacks (ACA). When there is a failure in one transaction and this leads to the rolling back or aborting other dependent transactions, then such scheduling is referred to as Cascading rollback or cascading abort. Example:

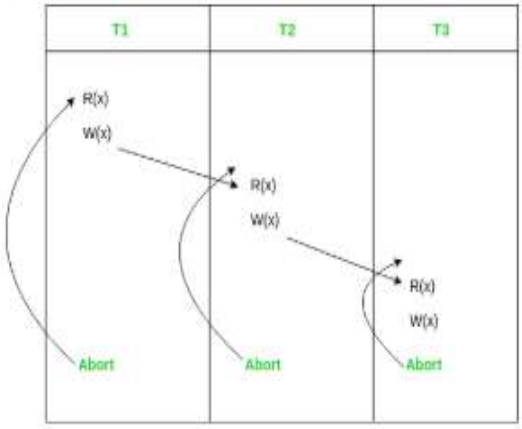



**Figure** - Cascading Abort

2. Cascadeless Schedule: Schedules in which transactions read values only after all transactions whose changes they are going to read commit are called cascadeless schedules. Avoids that a single transaction abort leads to a series of transaction rollbacks. A strategy to prevent cascading aborts is to disallow a transaction from reading uncommitted changes from another transaction in the same schedule. In other words, if some transaction $T_j$ wants to read value updated or written by some other transaction $T_i$, then the commit of $T_j$ must read it after the commit of $T_i$. **Example:** Consider the following schedule involving two transactions $T_1$ and $T_2$.

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| | W(A) |
| commit | |
| | R(A) |
| | commit |

This schedule is cascadeless. Since the updated value of **A** is read by $T_2$ only after the updating transaction i.e. $T_1$ commits. **Example:** Consider the following schedule involving two transactions $T_1$ and $T_2$.

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| | R(A) |
| | W(A) |
| abort | |
| | abort |

It is a recoverable schedule but it does not avoid cascading aborts. It can be seen that if $T_1$ aborts, $T_2$ will have to be aborted too in order to maintain the correctness of the schedule as $T_2$ has already read the uncommitted value written by $T_1$.

3. **Strict Schedule:** A schedule is strict if for any two transactions $T_i$, $T_j$, if a write operation of $T_i$ precedes a conflicting operation of $T_j$ (either read or write), then the commit or abort event of $T_i$ also precedes that conflicting operation of $T_j$. In other words, $T_j$ can read or write updated or written value of $T_i$ only after $T_i$ commits/aborts. **Example:** Consider the following schedule involving two transactions $T_1$ and $T_2$.

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| | R(A) |
| W(A) | |
| commit | |
| | W(A) |
| | R(A) |
| | commit |

This is a strict schedule since $T_2$ reads and writes A which is written by $T_1$ only after the commit of $T_1$.

2. **Non-Recoverable Schedule:Example:** Consider the following schedule involving two transactions $T_1$ and $T_2$.

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| | W(A) |
| | R(A) |
| | commit |
| abort | |

$T_2$ read the value of A written by $T_1$, and committed. $T_1$ later aborted, therefore the value read by $T_2$ is wrong, but since $T_2$ committed, this schedule is **non-recoverable**.

**1. Deferred Update:** It is a technique for the maintenance of the transaction log files of the DBMS. It is also called NO-UNDO/REDO technique. It is used for the recovery of transaction failures that occur due to power, memory, or OS failures. Whenever any transaction is executed, the updates are not made immediately to the database. They are first recorded on the log file and then those changes are applied once the commit is done. This is called the "Re-doing" process. Once the rollback is done none of the changes are applied to the database and the changes in the log file are also discarded. If the commit is done before crashing the system, then after restarting the system the changes that have been recorded in the log file are thus applied to the database.

**2. Immediate Update:** It is a technique for the maintenance of the transaction log files of the DBMS. It is also called UNDO/REDO technique. It is used for the recovery of transaction failures that occur due to power, memory, or OS failures. Whenever any transaction is executed, the updates are made directly to the database and the log file is also maintained which contains both old and new values. Once the commit is done, all the changes get stored permanently in the database, and records in the log file are thus discarded. Once rollback is done the old values get restored in the database and all the changes made to the database are also discarded. This is called the "Un-doing" process. If the commit is done before crashing the system, then after restarting the system the changes are stored permanently in the database.

| Deferred Update | Immediate Update |
|---|---|
| In a deferred update, the changes are not applied immediately to the database. | In an immediate update, the changes are applied directly to the database. |
| The log file contains all the changes that are to be applied to the database. | The log file contains both old as well as new values. |
| In this method once rollback is done all the records of log file are discarded and no changes are applied to the database. | In this method once rollback is done the old values are restored to the database using the records of the log file. |
| Concepts of buffering and caching are used in deferred update method. | Concept of shadow paging is used in immediate update method. |
| The major disadvantage of this method is that it requires a lot of time for recovery in case of system failure. | The major disadvantage of this method is that there are frequent I/O operations while the transaction is active. |
| In this method of recovery, firstly the changes carried out by a transaction on the data are done in the log file and then applied to the database on commit. Here, the maintained record gets discarded **on rollback** and thus, not applied to the database. | In this method of recovery, the database gets directly updated after the changes made by the transaction and the log file keeps the old and new values. In the case of **rollback**, these records are used to restore old values. |

| Feature | Deferred Update | Immediate Update |
|---|---|---|
| Update timing | Updates occur after instruction execution | Updates occur during instruction execution |
| Processor speed | May be faster: update occurs after instruction execution, allowing for multiple updates to be performed at once | May be slower: update occurs during instruction execution, potentially causing the processor to stall |
| Complexity | More complex: requires additional instructions or mechanisms to handle updates after instruction execution | Less complex: updates occur immediately during instruction execution |
| Consistency | May result in temporary inconsistency between data in registers and memory | Data in registers and memory are always consistent |
| Flexibility | May be more flexible: allows for more complex data manipulations and algorithms | Less flexible: immediate updates can limit the range of data manipulations and algorithms that can be performed |

## 3.4 Shadow Paging:

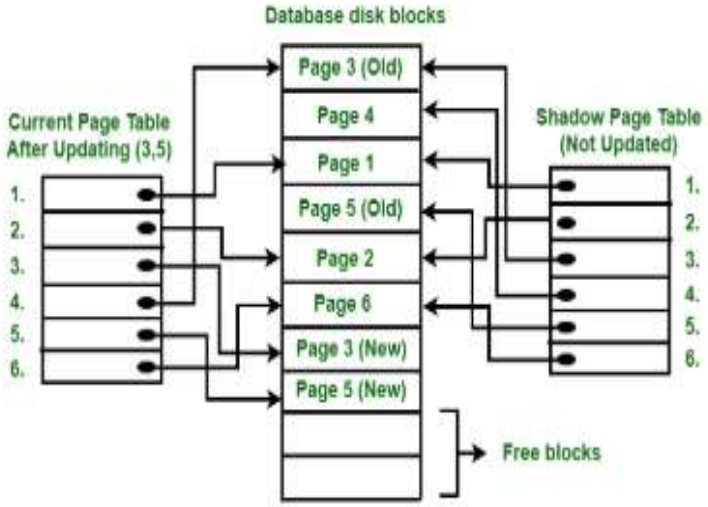
Shadow paging is one of the techniques that is used to recover from failure. We all know that recovery means to get back the information, which is lost. It helps to maintain database consistency in case of failure.

## Concept of shadow paging

Now let see the concept of shadow paging step by step –

- **Step 1** – Page is a segment of memory. Page table is an index of pages. Each table entry points to a page on the disk.
- **Step 2** – Two page tables are used during the life of a transaction: the current page table and the shadow page table. Shadow page table is a copy of the current page table.
- **Step 3** – When a transaction starts, both the tables look identical, the current table is updated for each write operation.
- **Step 4** – The shadow page is never changed during the life of the transaction.
- **Step 5** – When the current transaction is committed, the shadow page entry becomes a copy of the current page table entry and the disk block with the old data is released.
- **Step 6** – The shadow page table is stored in non-volatile memory. If the system crash occurs, then the shadow page table is copied to the current page table.

**Shadow Paging** is recovery technique that is used to recover database. In this recovery technique, database is considered as made up of fixed size of logical units of storage which are referred as **pages.** pages are mapped into physical blocks of storage, with help of the **page table** which allow one entry for each logical page of database. This method uses two page tables named **current page table** and **shadow page table**. The entries which are present in current page table are used to point to most recent database pages on disk. Another table i.e., Shadow page table is used when the transaction starts which is copying current page table. After this, shadow page table gets saved on disk and current page table is going to be used for transaction. Entries present in current page table may be changed during execution but in shadow page table it never get changed. After transaction, both tables become identical. This technique is also known as **Cut-of-Place updating.**




To understand concept, consider above figure. In this 2 write operations are performed on page 3 and 5. Before start of write operation on page 3, current page table points to old page 3. When write operation starts following steps are performed :

1. Firstly, search start for available free block in disk blocks.
2. After finding free block, it copies page 3 to free block which is represented by Page 3 (New).
3. Now current page table points to Page 3 (New) on disk but shadow page table points to old page 3 because it is not modified.
4. The changes are now propagated to Page 3 (New) which is pointed by current page table.

**COMMIT Operation :** To commit transaction following steps should be done :

1. All the modifications which are done by transaction which are present in buffers are transferred to physical database.
2. Output current page table to disk.
3. Disk address of current page table output to fixed location which is in stable storage containing address of shadow page table. This operation overwrites address of old shadow page table. With this current page table becomes same as shadow page table and transaction is committed.

**Failure :** If system crashes during execution of transaction but before commit operation, With this, it is sufficient only to free modified database pages and discard current page table. Before execution of transaction, state of database get recovered by reinstalling shadow page table. If the crash of system occur after last write operation then it does not affect propagation of changes that are made by transaction. These changes are preserved and there is no need to perform redo operation. **Advantages :**

- This method require fewer disk accesses to perform operation.
- In this method, recovery from crash is inexpensive and quite fast.
- There is no need of operations like- Undo and Redo.
- Recovery using this method will be faster.
- **Improved fault tolerance:** Shadow paging provides improved fault tolerance since it isolates transactions from each other. This means that if one transaction fails, it does not affect the other transactions that are currently executing.
- **Increased concurrency:** Since modifications made during a transaction are written to the shadow copy instead of the actual database, multiple transactions can be executed concurrently without interfering with each other. This leads to increased concurrency and better performance.
- **Simplicity:** Shadow paging is a relatively simple technique to implement. It requires minimal modifications to the existing database system, making it easier to integrate into existing systems.
- **No need for log files:** In traditional database systems, log files are used to maintain a record of all changes made to the database. Shadow paging eliminates the need for log files since all changes are made to the shadow copy. This reduces the overhead associated with maintaining log files and makes the system more efficient.

**Disadvantages :**

- Due to location change on disk due to update database it is quite difficult to keep related pages in database closer on disk.
- During commit operation, changed blocks are going to be pointed by shadow page table which have to be returned to collection of free blocks otherwise they become accessible.
- The commit of single transaction requires multiple blocks which decreases execution speed.
- To allow this technique to multiple transactions concurrently it is difficult.
- **Data fragmentation:** The main disadvantage of this technique is the updated Data will suffer from fragmentation as the data is divided up into pages that may or not be in linear order for large sets of related hence, complex storage management strategies.
- **Garbage collection:** Garbage will accumulate in the pages on the disk as data is updated and pages lose any references. For example if i have a page that contains a data item X that is replaced with a new value then a new page will be created. Once the shadow page table is updated nothing will reference the old value of X. The operation to migrate between current and shadow directories must be implemented as an atomic mode.
- **Performance overhead:** Since modifications made during a transaction are written to the shadow copy, there is a performance overhead associated with copying the changes back to the actual database once the transaction is committed. This can impact the overall performance of the system.
- **Limited concurrency control:** Shadow paging does not provide strong concurrency control mechanisms. While it allows for multiple transactions to execute concurrently, it does not prevent conflicts between transactions. This means that transactions can interfere with each other, leading to inconsistencies in the database.
- **Difficult to implement for some systems:** Shadow paging can be difficult to implement for some systems that have complex data structures or use a lot of shared memory. In these cases, it may not be possible to maintain a shadow copy of the entire database.
- **Limited fault tolerance:** While shadow paging does provide improved fault tolerance in some cases, it does not provide complete fault tolerance. In the event of a crash, there is still a risk of data loss if the changes made during a transaction are not properly copied to the actual database.

## 3.5 ARIES Algorithm:

The Aries Recovery Algorithm in DBMS is an important method for maintaining data integrity and consistency, particularly after a system crash. Aries, which stands for Algorithm for Recovery and Isolation Exploiting Semantics, employs a mix of logging, checkpoints, and transaction recovery to effectively manage the database's state.

Key Features of Aries Recovery Algorithm in DBMS

1. **Write-Ahead Logging (WAL)**: This makes sure that all changes are logged before they are applied to the database.
2. **CheckPointing**: To create a stable point in the database from which recovery can start.
3. **Three Phases of Recovery**: To ensure database recovery Analysis, Redo, and Undo phases are crucial.

**Aries Recovery Algorithm Phases**

**1. Analysis Phase**

The Analysis Phase identifies the database's state and determines which log entries need to be processed for recovery.

**Example:** Suppose we have a log with the following entries:

1. **<T1, Start>**
2. **<T1, Write(A, 10)>**
3. **<T1, Commit>**
4. **<T2, Start>**
5. **<T2, Write(B, 20)>**
6. **<T2, Abort>**

During the Analysis Phase:

- The system reads the log to identify active transactions (T2) and committed transactions (T1).
- It also records the state of database pages that were modified by active transactions.

**Steps:**

1. Scan the log to identify transactions and their states.
2. Create a list of dirty pages (pages that have been modified but not yet written to disk).

**2. Redo Phase**

The Redo Phase re-applies changes recorded in the log to ensure that all committed transactions are properly reflected in the database.

**Example:** Using the log from the previous example, the Redo Phase will:

- Reapply the changes made by T1 and T2. Since T1 committed, its changes to page A will be redone.
- T2's changes are ignored because the transaction was aborted.

**Steps:**
1.Replay every action taken during a committed transaction.
2.Ensure all modifications are applied to the database pages as per the log.

**3. Undo Phase**
The Undo Phase reverses changes made by aborted transactions, restoring the database to a consistent state by undoing any modifications from these transactions.

**Example:** From the log, we need to undo changes made by T2.

**Steps:**
1.Identify all operations from aborted transactions.
2.Apply undo operations to revert changes made by these transactions.

**Example of Aries Recovery Algorithm in DBMS**
Let's consider a more detailed scenario:

**Initial State**:
- Page A: 0
- Page B: 0

**Transactions**:
- **T1** writes A = 10 and commits.
- **T2** writes B = 20 and aborts.

**Log Entries**
- **<T1, Start>**
- **<T1, Write(A, 10)>**
- **<T1, Commit>**
- **<T2, Start>**
- **<T2, Write(B, 20)>**
- **<T2, Abort>**

**After Crash**:
- Checkpoint indicates page A was modified.
- Analysis Phase finds T1 committed, T2 aborted.
- Redo Phase reapplies T1's write to page A.
- Undo Phase rolls back T2's write to page B.

**Advantages of Aries Recovery Algorithm in DBMS**

**1. Fine-Grained Locking**
ARIES uses fine-grained locking, allowing multiple transactions to access different parts of the same data item at the same time. This increases database concurrency, leading to better resource use and improved performance.

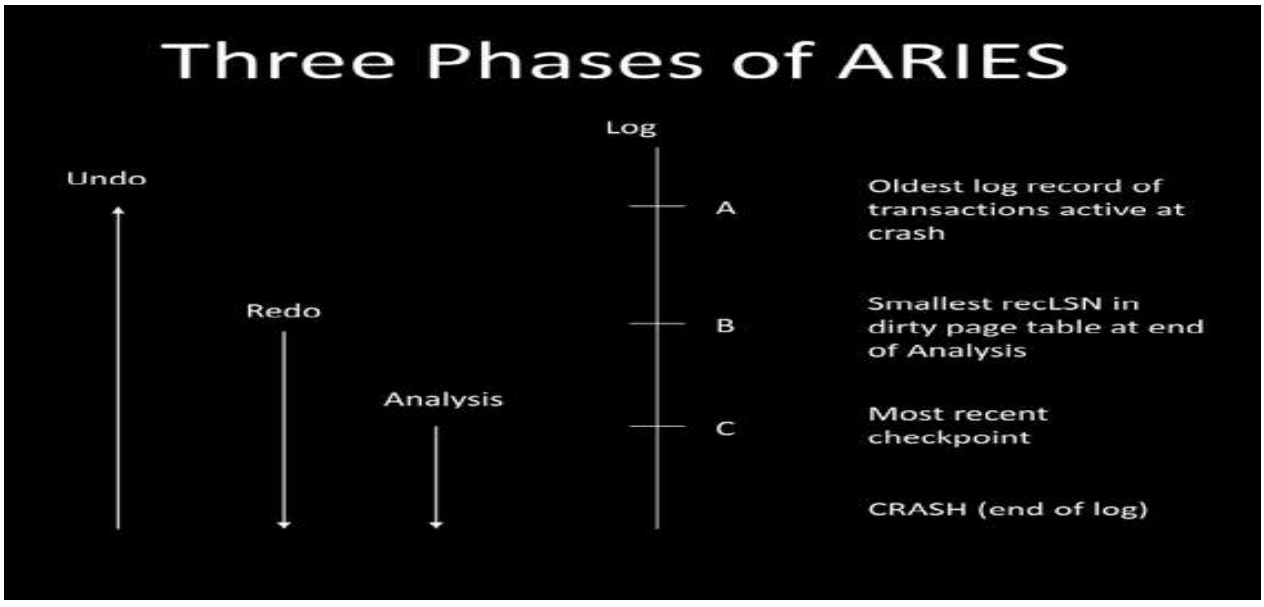
**2. Efficient Crash Recovery**
ARIES handles crash recovery efficiently through three phases: Analysis, Redo, and Undo:
**Analysis**: Finds the checkpoint where recovery should start and identifies the transactions that were active during the crash.
- **Redo**: Reapplies all updates from the log to ensure the database reflects all committed transactions, even if they were not saved to disk before the crash.
- **Undo**: Reverts changes from incomplete transactions at the time of the crash to keep the database consistent.

**3. Logical Logging**
ARIES mainly uses physical logging, which records the before and after images of database pages. It can also use logical logging when needed. Logical logging records higher-level operations, which can help with semantic optimizations and might reduce the amount of data logged and processed during recovery.

**4. Non-Locking Concurrency Controls**
ARIES supports non-locking concurrency controls, such as optimistic concurrency control and timestamp-based methods. These methods can improve performance in situations where conflicts are rare and rollback costs are low.

**5. Incremental Recovery**
The algorithm allows for incremental or fuzzy recovery of the database, enabling the database to be partially available even while recovery is ongoing. This is critical for large databases and enterprise applications where downtime must be minimized.

**6. Support for Multiple Granularity Locking**
ARIES can handle locks of multiple granularities (e.g., row-level, page-level, table-level), which allows it to be adaptable to various application needs, balancing between concurrency and locking overhead.



## ARIES Algorithm

• ARIES is a recovery algorithm.

• It stands for Algorithm for Recovery and Isolation Exploiting Semantics.

• It is based on Write Ahead Log (WAL) protocol.

• When database crashes during some transaction processing, we have the logs that got saved to the disk.

• ARIES has 3 phases that occur in the following order -

**1) Analysis:**

• Scan the log from start to reconstruct the transaction and dirty page table. Dirtypages contain data that has been changed but not yet written to disk.

• The active transactions which were present at the time of crash are identified.

• During analysis phase the log is scanned forward from the checkpoint record to construct snapshot of what system looks like at the time of crash.

**2) Redo :**

• This phase is started only after completion of analysis phase.

• The log is read forward and each update is redone.

**3) Undo :**

• This phase is started after redo phase.

• The log is scanned backward and updates to corresponding active transactions are undone.

**Advantages:**

1) It is simple and flexible.

2) It supports concurrency control protocol.

3) Independent recovery of every page.

## 3.6 Serializability:

A schedule is serialized if it is equivalent to a serial schedule. A concurrent schedule must ensure it is the same as if executed serially means one after another. It refers to the sequence of actions such as read, write, abort, commit are performed in a serial manner.

**Types of Serializability:**
There are two types of Serializability:

**i) View Serializability**
A schedule is view-Serializability if it is viewed equivalent to a serial schedule.

The rules it follows are as follows:

- T1 is reading the initial value of A, then T2 also reads the initial value of A.
- T1 is the reading value written by T2, then T2 also reads the value written by T1.

- T1 is writing the final value, and then T2 also has the write operation as the final value.

**ii) Conflict Serializability**

It orders any conflicting operations in the same way as some serial execution. A pair of operations is said to conflict if they operate on the same data item and one of them is a write operation.

That means

- Read i(x) read j(x) - non-conflict read-read operation
- Read i(x) write j(x) - conflict read-write operation.
- Write i(x) read j(x) - conflict write-read operation.

Write i(x) write j(x) - conflict write-write operation.

## Types of Serializability

There are two ways to check whether any non-serial schedule is serializable.




*Types of Serializability - Conflict & View*

**1. Conflict serializability**

Conflict serializability refers to a subset of serializability that focuses on maintaining the consistency of a database while ensuring that identical data items are executed in an order. In a DBMS each transaction has a value and all the transactions, in the database rely on this uniqueness. This uniqueness ensures that no two operations with the conflict value can occur simultaneously.

For example lets consider an order table and a customer table as two instances. Each order is associated with one customer even though a single client may place orders. However there are restrictions for achieving conflict serializability in the database. Here are a few of them.

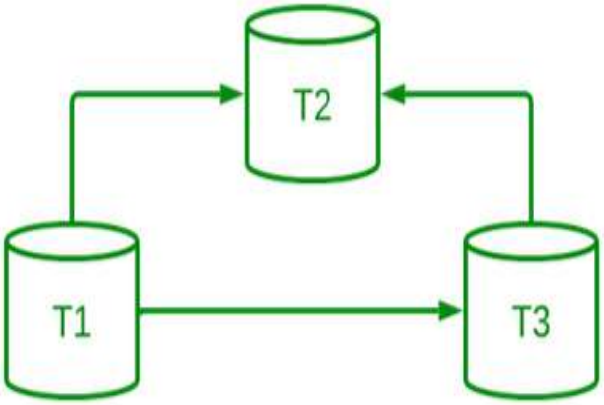
1. Different transactions should be used for the two procedures.
2. The identical data item should be present in both transactions.
3. Between the two operations, there should be at least one write operation.

**Example**

Three transactions—t1, t2, and t3—are active on a schedule "S" at once. Let's create a graph of precedence.

| Transaction - 1 (t1) | Transaction - 2 (t2) | Transaction - 3 (t3) |
|---|---|---|
| R(a) | | |
| | R(b) | |
| | | R(b) |
| | W(b) | |
| W(a) | | |
| | | W(a) |
| | R(a) | |
| | W(a) | |

It is a conflict serializable schedule as well as a serial schedule because the graph (a DAG) has no loops. We can also determine the order of transactions because it is a serial schedule.




*DAG of transactions*

As there is no incoming edge on Transaction 1, Transaction 1 will be executed first. T3 will run second because it only depends on T1. Due to its dependence on both T1 and T3, t2 will finally be executed.

Therefore, the serial schedule's equivalent order is: t1 --> t3 --> t2

**Note:** A schedule is unquestionably consistent if it is conflicting serializable. A non-conflicting serializable schedule, on the other hand, might or might not be serial. We employ the idea of View Serializability to further examine its serial behavior.

## 2. View Serializability

View serializability is a kind of operation in a serializable in which each transaction should provide some results, and these outcomes are the output of properly sequentially executing the data item. The view serializability, in contrast to conflict serialized, is concerned with avoiding database inconsistency. The view serializability feature of DBMS enables users to see databases in contradictory ways.

To further understand view serializability in DBMS, we need to understand the schedules S1 and S2. The two transactions T1 and T2 should be used to establish these two schedules. Each schedule must follow the three transactions in order to retain the equivalent of the transaction. These three circumstances are listed below.

1. The first prerequisite is that the same kind of transaction appears on every schedule. This requirement means that the same kind of group of transactions cannot appear on both schedules S1 and S2. The schedules are not equal to one another if one schedule commits a transaction but it does not match the transaction of the other schedule.
2. The second requirement is that different read or write operations should not be used in either schedule. On the other hand, we say that two schedules are not similar if schedule S1 has two write operations whereas schedule S2 only has one. The number of the write operation must be the same in both schedules, however there is no issue if the number of the read operation is different.
3. The second to last requirement is that there should not be a conflict between either timetable. execution order for a single data item. Assume, for instance, that schedule S1's transaction is T1, and schedule S2's transaction is T2. The data item A is written by both the transaction T1 and the transaction T2. The schedules are not equal in this instance. However, we referred to the schedule as equivalent to one another if it had the same number of all write operations in the data item.

### What is view equivalency?

Schedules (S1 and S2) must satisfy these two requirements in order to be viewed as equivalent:

1. The same piece of data must be read for the first time. For instance, if transaction t1 is reading "A" from the database in schedule S1, then t1 must also read A in schedule S2.
2. The same piece of data must be used for the final write. As an illustration, if transaction t1 updated A last in S1, it should also conduct final write in S2.
3. The middle sequence need to follow suit. As an illustration, if in S1 t1 is reading A, and t2 updates A, then in S2 t1 should read A, and t2 should update A.

View Serializability refers to the process of determining whether a schedule's views are equivalent.

**Example**

We have a schedule "S" with two concurrently running transactions, "t1" and "t2."

**Schedule - S:**

| Transaction-1 (t1) | Transaction-2 (t2) |
|---|---|
| R(a) | |
| W(a) | |
| | R(a) |
| | W(a) |
| R(b) | |
| W(b) | |
| | R(b) |
| | W(b) |

By switching between both transactions' mid-read-write operations, let's create its view equivalent schedule (S').

**Schedule - S':**

| Transaction-1 (t1) | Transaction-2 (t2) |
|---|---|
| R(a) | |
| W(a) | |
| R(b) | |
| W(b) | |
| | R(a) |
| | W(a) |
| | R(b) |
| | W(b) |

It is a view serializable schedule since a view similar schedule is conceivable.

**Note:** A conflict serializable schedule is always viewed as serializable, but vice versa is not always true.

### Advantages of Serializability

1. **Execution is predictable:** In serializable, the DBMS's threads are all performed simultaneously. The DBMS doesn't include any such surprises. In DBMS, no data loss or corruption occurs and all variables are updated as intended.
2. DBMS executes each thread independently, making it much simpler to understand and troubleshoot each database thread. This can greatly simplify the debugging process. The concurrent process is therefore not a concern for us.
3. **Lower Costs:** The cost of the hardware required for the efficient operation of the database can be decreased with the aid of the serializable property. It may also lower the price of developing the software.
4. **Increased Performance:** Since serializable executions provide developers the opportunity to optimize their code for performance, they occasionally outperform non-serializable equivalents.

## Types of Serializability

In DBMS, all the transaction should be arranged in a particular order, even if all the transaction is concurrent. If all the transaction is not serializable, then it produces the incorrect result.

In DBMS, there are different types of serializable. Each type of serializable has some advantages and disadvantages. The two most common types of serializable are view serializability and conflict serializability.

### 1. Conflict Serializability

Conflict serializability is a type of conflict operation in serializability that operates the same data item that should be executed in a particular order and maintains the consistency of the database. In DBMS, each transaction has some unique value, and every transaction of the database is based on that unique value of the database.

This unique value ensures that no two operations having the same conflict value are executed concurrently. For example, let's consider two examples, i.e., the order table and the customer table. One customer can have multiple orders, but each order only belongs to one customer. There is some condition for the conflict serializability of the database. These are as below.

- Both operations should have different transactions.
- Both transactions should have the same data item.
- There should be at least one write operation between the two operations.

If there are two transactions that are executed concurrently, one operation has to add the transaction of the first customer, and another operation has added by the second operation. This process ensures that there would be no inconsistency in the database.

### 2. View Serializability

View serializability is a type of operation in the serializable in which each transaction should produce some result and these results are the output of proper sequential execution of the data item. Unlike conflict serialized, the view serializability focuses on preventing inconsistency in the database. In DBMS, the view serializability provides the user to view the database in a conflicting way.

In DBMS, we should understand schedules S1 and S2 to understand view serializability better. These two schedules should be created with the help of two transactions T1 and T2. To maintain the equivalent of the transaction each schedule has to obey the three transactions. These three conditions are as follows.

- The first condition is each schedule has the same type of transaction. The meaning of this condition is that both schedules S1 and S2 must not have the same type of set of transactions. If one schedule has committed the transaction but does not match the transaction of another schedule, then the schedule is not equivalent to each other.
- The second condition is that both schedules should not have the same type of read or write operation. On the other hand, if schedule S1 has two write operations while schedule S2 has one write operation, we say that both schedules are not equivalent to each other. We may also say that there is no problem if the number of the read operation is different, but there must be the same number of the write operation in both schedules.
- The final and last condition is that both schedules should not have the same conflict. Order of execution of the same data item. For example, suppose the transaction of schedule S1 is T1, and the transaction of schedule S2 is T2. The transaction T1 writes the data item A, and the transaction T2 also writes the data item A. in this case, the schedule is not equivalent to each other. But if the schedule has the same number of each write operation in the data item then we called the schedule equivalent to each other.

## 3.7 Transaction Support in SQL:

**SQL transactions** are essential for ensuring **data integrity** and consistency in **relational databases**. Transactions allow for a group of **SQL operations** to be executed as a **single unit**, ensuring that either all the operations succeed or none of them do. This capability, together with the **ACID** (Atomicity, Consistency, Isolation, and Durability) properties, is fundamental for maintaining the **integrity of our data.**

### The SQL Transaction Process

Transactions are controlled using specific SQL commands. The most common commands used for managing transactions are:

1. **BEGIN TRANSACTION**: Starts a transaction.
2. **COMMIT**: Commits the transaction, saving all changes made.
3. **ROLLBACK**: Rolls back the transaction, undoing all changes made since the `BEGIN TRANSACTION`.
4. **SAVEPOINT**: Marks a point within a transaction to which you can later roll back.

### Example of SQL Transaction with a Bank Transfer Scenario

Let's look at an example of a bank transfer between two accounts. This example demonstrates the usage of multiple queries in a single transaction.

```
BEGIN TRANSACTION;

-- Deduct $150 from Account A
UPDATE Accounts
SET Balance = Balance - 150
WHERE AccountID = 'A';

-- Add $150 to Account B
UPDATE Accounts
SET Balance = Balance + 150
WHERE AccountID = 'B';

-- Commit the transaction if both operations succeed
COMMIT;
```

If any error occurs, such as an issue with the `UPDATE` query, you can use `ROLLBACK` to undo all changes made during the transaction:

```
ROLLBACK;
```

This ensures that the system doesn't end up in an inconsistent state, such as deducting money from one account without adding it to another.

## SQL Transaction Control Commands

In SQL, transaction control commands manage the **execution of SQL operations**, ensuring the integrity and reliability of **database transactions**. These commands help manage the start, commit, and rollback of changes made to the database. Below are the **key transaction control commands** in SQL, explained with syntax and examples for each.

### 1. BEGIN TRANSACTION Command

The `BEGIN TRANSACTION` command marks the beginning of a new transaction. All SQL statements that follow this command will be part of the same transaction until a `COMMIT` or `ROLLBACK` is encountered. This command doesn't make any changes to the database, it just starts the transaction.

**Syntax:**

```
BEGIN TRANSACTION transaction_name ;
```

**Example**

```
BEGIN TRANSACTION TransferFunds;
```

### 2. COMMIT Command

The `COMMIT` command is used to save all changes made during the current transaction to the database. Once a transaction is committed, the changes are permanent.

**Syntax:**

```
COMMIT;
```

Here is the sample `Student` table that will be used to perform the operations in this example. This table contains basic student details such as **ID**, **name**, **age**, and other relevant information that will be manipulated using various **transaction control commands**.

| Student | | | | |
|---|---|---|---|---|
| Rol_No | Name | Address | Phone | Age |
| 1 | Ram | Delhi | 9455123451 | 18 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |
| 3 | Sujit | Rohtak | 9156253131 | 20 |
| 4 | Suresh | Delhi | 9156768971 | 18 |
| 3 | Sujit | Rohtak | 9156253131 | 20 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |

*Student Table*

**Example**

Following is an example which would delete those records from the table which have age = 20 and then COMMIT the changes in the database.

```
DELETE FROM Student WHERE AGE = 20;
COMMIT;
```

**Output**

| Rol_No | Name | Address | Phone | Age |
|---|---|---|---|---|
| 1 | Ram | Delhi | 9455123451 | 18 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |
| 4 | Suresh | Delhi | 9156768971 | 18 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |

*output*

### 3. ROLLBACK Command

The **ROLLBACK command** is used to undo all changes made in the current transaction. It is used when an error occurs or when the desired changes cannot be completed. The database will revert to the state it was in before the BEGIN TRANSACTION was executed.

**Syntax:**

```
ROLLBACK;
```

**Example**

Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database. In this case, the DELETE operation is undone, and the changes to the database are not saved.

```
DELETE FROM Student WHERE AGE = 20;
ROLLBACK;
```

**Output**

| Student | | | | |
|---|---|---|---|---|
| Rol_No | Name | Address | Phone | Age |
| 1 | Ram | Delhi | 9455123451 | 18 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |
| 3 | Sujit | Rohtak | 9156253131 | 20 |
| 4 | Suresh | Delhi | 9156768971 | 18 |
| 3 | Sujit | Rohtak | 9156253131 | 20 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |

*output*

### 4. SAVEPOINT Command

A SAVEPOINT is used to create a checkpoint within a transaction. We can roll back to a specific SAVEPOINT instead of rolling back the entire transaction. This allows us to **undo part** of the transaction **rather than** the entire transaction.

**Syntax:**

```
SAVEPOINT SAVEPOINT_NAME;
```

**Example**

```
SAVEPOINT SP1;
//Savepoint created.
DELETE FROM Student WHERE AGE = 20;
//deleted
SAVEPOINT SP2;
//Savepoint created.
```

**Output**

| Rol_No | Name | Address | Phone | Age |
|---|---|---|---|---|
| 1 | Ram | Delhi | 9455123451 | 18 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |
| 4 | Suresh | Delhi | 9156768971 | 18 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |

*output*

**Explanation:**

From the above example **Sample table1**, Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database by keeping Savepoints. Here SP1 is first SAVEPOINT created before deletion. In this example one deletion have taken place. After deletion again SAVEPOINT SP2 is created.

## 5. ROLLBACK TO SAVEPOINT

The `ROLLBACK TO SAVEPOINT` command allows us to roll back the transaction to a specific savepoint, effectively undoing changes made after that point.

**Syntax:**

```
ROLLBACK TO SAVEPOINT SAVEPOINT_NAME;
```

**Example**

Deletion have been taken place, let us assume that we have changed our mind and decided to ROLLBACK to the SAVEPOINT that we identified as SP1 which is before deletion. So, In this case the `DELETE` operation is undone, and the transaction is returned to the state it was in at the `SP1` savepoint.

```
ROLLBACK TO SP1;
//Rollback completed
```

**Output**

| Student | | | | |
|---|---|---|---|---|
| Rol_No | Name | Address | Phone | Age |
| 1 | Ram | Delhi | 9455123451 | 18 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |
| 3 | Sujit | Rohtak | 9156253131 | 20 |
| 4 | Suresh | Delhi | 9156768971 | 18 |
| 3 | Sujit | Rohtak | 9156253131 | 20 |
| 2 | Ramesh | Gurgaon | 9652431543 | 18 |

*output*

## 6. RELEASE SAVEPOINT Command

This command is used to remove a **SAVEPOINT** that we have created. Once a SAVEPOINT has been released, we can no longer use the **ROLLBACK command** to undo transactions performed since the last SAVEPOINT. It is used to initiate a **database transaction** and used to specify characteristics of the transaction that follows.

**Syntax:**

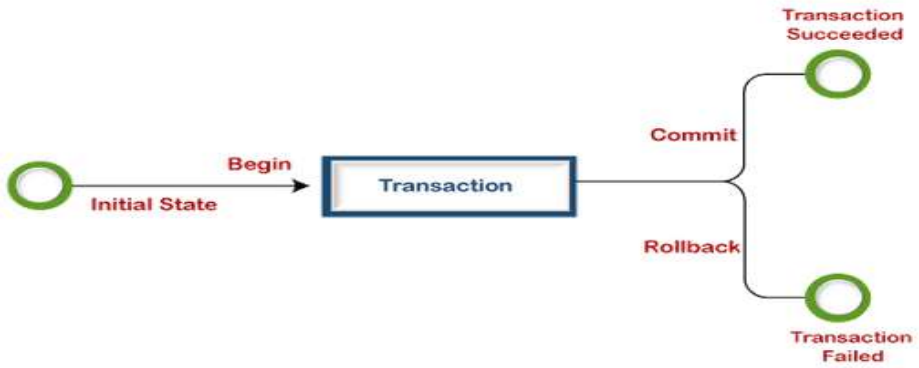
```
RELEASE SAVEPOINT SAVEPOINT_NAME
```

**Example**

Once the savepoint `SP2` is released, we can no longer roll back to it.

```
RELEASE SAVEPOINT SP2; -- Release the second savepoint.
```

The below pictorial representation explains the transaction process:



**Summary of Transaction Control Commands**

| Command | Purpose | Example |
|---|---|---|
| **BEGIN TRANSACTION** | Starts a new transaction. | `BEGIN TRANSACTION TransferFunds;` |
| **COMMIT** | Saves all changes made during the transaction. | `COMMIT;` |
| **ROLLBACK** | Undoes changes made during the transaction. | `ROLLBACK;` |
| **SAVEPOINT** | Creates a point in the transaction to which you can roll back. | `SAVEPOINT SP1;` |
| **ROLLBACK TO SAVEPOINT** | Rolls back the transaction to a specific savepoint. | `ROLLBACK TO SP1;` |
| **RELEASE SAVEPOINT** | Removes a savepoint from the transaction. | `RELEASE SAVEPOINT SP2;` |

### Types of SQL Transactions

There are different types of transactions based on their nature and the specific operations they perform:

- **Read Transactions**: Used to only read the data, typically with `SELECT` queries.
- **Write Transactions**: These involve modifying the data in the database with `INSERT`, `UPDATE`, or `DELETE` operations.
- **Distributed Transactions**: These transactions span multiple databases and ensure consistency across them.
- **Implicit Transactions**: Automatically started by SQL Server for certain operations.
- **Explicit Transactions**: Manually controlled transactions where the user begins and ends the transaction using `BEGIN TRANSACTION`, `COMMIT`, and `ROLLBACK`.

## How to Monitor and Optimize SQL Transactions

Optimizing SQL transactions is critical for performance. Here are some best practices:

- **Keep Transactions Short**: Avoid long-running transactions as they can lock tables and reduce concurrency.
- **Use Proper Indexing**: Proper indexing on tables involved in the transaction can greatly improve performance.
- **Avoid Transaction Blocking**: Ensure transactions don't block others unnecessarily, causing delays.
- **Commit Early**: Try to commit as soon as the transaction is complete to release resources.

## Conclusion

**SQL transactions** are a fundamental part of **database management**, providing a way to ensure **data consistency**, integrity, and isolation. Understanding how to use **`BEGIN TRANSACTION`**, **`COMMIT`**, **`ROLLBACK`**, and `SAVEPOINT` is essential for writing reliable and efficient SQL queries. With proper transaction control, we can ensure that our database operations are **robust**, **consistent**, and optimized for performance. By following best practices like keeping transactions short and committing early, we can also improve the **scalability** and concurrency of our database.

## 3.8 Need for Concurrency:

Following are the purposes of concurrency control -

- To ensure isolation
- To resolve read-write or write-write conflicts
- To preserve consistency of database
- Concurrent execution of transactions over shared database creates several data integrity and consistency problems - these are

**(1) Lost update problem:** This problem occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.

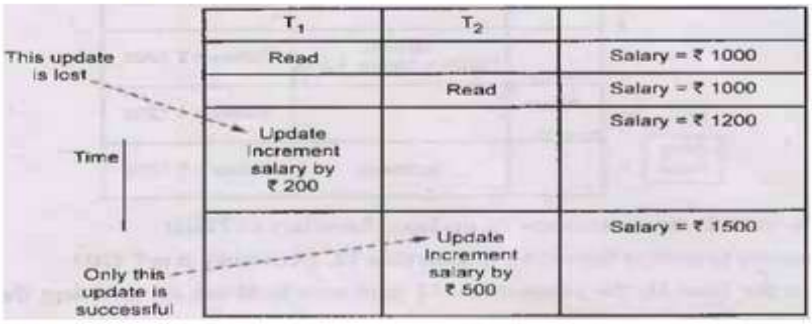
For example - Consider following transactions

(1) Salary of Employee is read during transaction T1.

(2) Salary of Employee is read by another transaction T2.

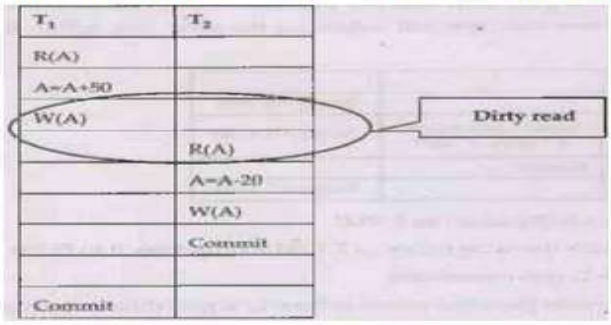
(3) During transaction T1, the salary is incremented by 200

(4) During transaction T2, the salary is incremented by 500



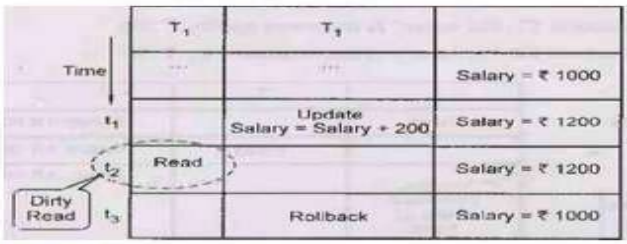

The result of the above sequence is that the update made by transaction T1 is completely lost. Therefor this problem is called as lost update problem.

**(2) Dirty read or Uncommited read problem:** The dirty read is a situation in which one transaction reads the data immediately after the write operation of previous transaction




For example - Consider following transactions -

Assume initially salary is = 1000




(1) At the time t1, the transaction T2 updates the salary to 1200

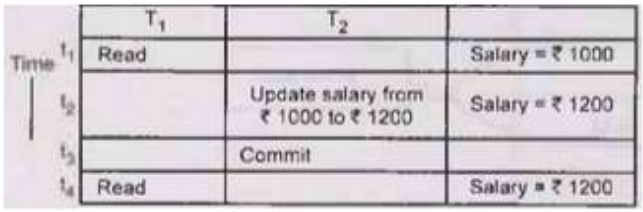
(2) This salary is read at time t2 by transaction T1. Obviously it is 1200

(3) But at the time t3, the transaction T2 performs Rollback by undoing the changes made by T1 and T2 at time t1 and t2.

(4) Thus the salary again becomes = 1000. This situation leads to Dirty Read or Uncommited Read because here the read made at time t2(immediately after roid update of another transaction) becomes a dirty read.

**(3) Non-repeatable read problem**

This problem is also known as inconsistent analysis problem. This problem occurs when a particular transaction sees two different values for the same row within its lifetime. For example-



| Time | $T_1$ | $T_2$ | |
|---|---|---|---|
| $t_1$ | Read | | Salary = ₹ 1000 |
| $t_2$ | | Update salary from ₹ 1000 to ₹ 1200 | Salary = ₹ 1200 |
| $t_3$ | | Commit | |
| $t_4$ | Read | | Salary = ₹ 1200 |

(1) At time $t_1$, the transaction $T_1$ reads the salary as 1000

(2) At time $t_2$ the transaction $T_2$ reads the same salary as 1000 and updates it to 1200

(3) Then at time $t_3$, the transaction $T_2$ gets committed.

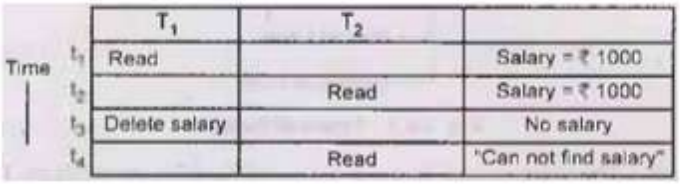
(4) Now when the transaction $T_1$ reads the same salary at time $t_4$, it gets different value than what it had read at time $t_1$. Now, transaction $T_1$, cannot repeat its reading operation. Thus inconsistent values are obtained.

Hence the name of this problem is non-repeatable read or inconsistent analysis problem.

**(4) Phantom read problem**

The phantom read problem is a special case of non repeatable read problem.

This is a problem in which one of the transaction makes the changes in the database system and due to these changes another transaction can not read the data item which it has read just recently. For example -



| Time | $T_1$ | $T_2$ | |
|---|---|---|---|
| $t_1$ | Read | | Salary = ₹ 1000 |
| $t_2$ | | Read | Salary = ₹ 1000 |
| $t_3$ | Delete salary | | No salary |
| $t_4$ | | Read | "Can not find salary" |

(1) At time $t_1$, the transaction $T_1$ reads the value of salary as 1000

(2) At time $t_2$, the transaction $T_2$ reads the value of the same salary as 1000

(3) At time $t_3$, the transaction $T_1$ deletes the variable salary.

(4) Now at time $t_4$, when $T_2$ again reads the salary it gets error. Now transaction $T_2$ can not identify the reason why it is not getting the salary value which is read just few time back.

This problem occurs due to changes in the database and is called phantom read problem.

Concurrency control is an essential aspect of database management systems (DBMS) that ensures transactions can execute concurrently without interfering with each other. However, concurrency control can be challenging to implement, and without it, several problems can arise, affecting the consistency of the database. In this article, we will discuss some of the concurrency problems that can occur in DBMS transactions and explore solutions to prevent them.

When multiple transactions execute concurrently in an uncontrolled or unrestricted manner, then it might lead to several problems. These problems are commonly referred to as concurrency problems in a database environment.

The five concurrency problems that can occur in the database are:

- Temporary Update Problem
- Incorrect Summary Problem
- Lost Update Problem
- Unrepeatable Read Problem
- Phantom Read Problem

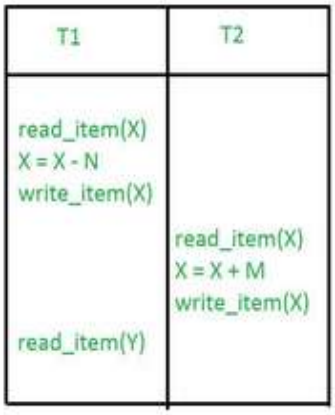
Concurrency control ensures the consistency and integrity of data in databases when multiple transactions are executed simultaneously. Understanding issues like lost updates, dirty reads, and non-repeatable reads is crucial when studying DBMS.

These are explained as following below.

### Temporary Update Problem:

Temporary update or dirty read problem occurs when one transaction updates an item and fails. But the updated item is used by another transaction before the item is changed or reverted back to its last value.

**Example:**




In the above example, if transaction 1 fails for some reason then X will revert back to its previous value. But transaction 2 has already read the incorrect value of X.

### Incorrect Summary Problem:

Consider a situation, where one transaction is applying the aggregate function on some records while another transaction is updating these records. The aggregate function may calculate some values before the values have been updated and others after they are updated.

**Example:**

| T1 | T2 |
|---|---|
| | sum = 0<br>read_item(A)<br>sum = sum + A |
| read_item(X)<br>X = X - N<br>write_item(X) | |
| | read_item(X)<br>sum = sum + X<br>read_item(Y)<br>sum = sum + Y |
| read_item(Y)<br>Y = Y + N<br>write_item(Y) | |

In the above example, transaction 2 is calculating the sum of some records while transaction 1 is updating them. Therefore the aggregate function may calculate some values before they have been updated and others after they have been updated.

### Lost Update Problem:

In the lost update problem, an update done to a data item by a transaction is lost as it is overwritten by the update done by another transaction.

**Example:**

| T1 | T2 |
|---|---|
| read_item(X)<br>X = X + N | |
| | X = X + 10<br>write_item(X) |

In the above example, transaction 2 changes the value of X but it will get overwritten by the write commit by transaction 1 on X *(not shown in the image above)*. Therefore, the update done by transaction 2 will be lost. Basically, the write commit done by the **last transaction** will overwrite all previous write commits.

### Unrepeatable Read Problem:

The unrepeatable problem occurs when two or more read operations of the same transaction read different values of the same variable.

**Example:**

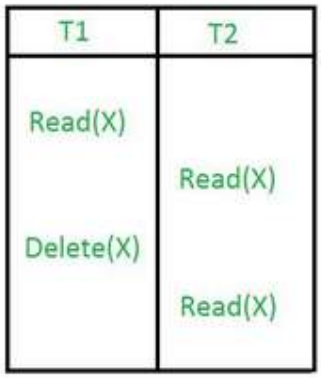
| T1 | T2 |
|---|---|
| Read(X) | |
| | Read(X) |
| Write(X) | |
| | Read(X) |

In the above example, once transaction 2 reads the variable X, a write operation in transaction 1 changes the value of the variable X. Thus, when another read operation is performed by transaction 2, it reads the new value of X which was updated by transaction 1.

**Phantom Read Problem:**

The phantom read problem occurs when a transaction reads a variable once but when it tries to read that same variable again, an error occurs saying that the variable does not exist.

**Example:**




In the above example, once transaction 2 reads the variable X, transaction 1 deletes the variable X without transaction 2's knowledge. Thus, when transaction 2 tries to read X, it is not able to do it.

## Solution :

To prevent concurrency problems in DBMS transactions, several concurrency control techniques can be used, including locking, timestamp ordering, and optimistic concurrency control.

Locking involves acquiring locks on the data items used by transactions, preventing other transactions from accessing the same data until the lock is released. There are different types of locks, such as shared and exclusive locks, and they can be used to prevent Dirty Read and Non-Repeatable Read.

Timestamp ordering assigns a unique timestamp to each transaction and ensures that transactions execute in timestamp order. Timestamp ordering can prevent Non-Repeatable Read and Phantom Read.

Optimistic concurrency control assumes that conflicts between transactions are rare and allows transactions to proceed without acquiring locks initially. If a conflict is detected, the transaction is rolled back, and the conflict is resolved. Optimistic concurrency control can prevent Dirty Read, Non-Repeatable Read, and Phantom Read.

In conclusion, concurrency control is crucial in DBMS transactions to ensure data consistency and prevent concurrency problems such as Dirty Read, Non-Repeatable Read, and Phantom Read. By using techniques like locking, timestamp ordering, and optimistic concurrency control, developers can build robust database systems that support concurrent access while maintaining data consistency.

## 3.9 Concurrency Control:

Concurrency control is a very important concept of DBMS which ensures the simultaneous execution or manipulation of data by several processes or user without resulting in data inconsistency. Concurrency Control deals with **interleaved execution** of more than one transaction.

The concept of a transaction has been applied broadly in database systems and applications. While the initial use of transactions was in financial applications, the concept is now used in real-time applications in telecommunication, as well as in the management of long-duration activities such as product design or administrative workflows.

A set of logically related operations is known as a transaction. The main operations of a transaction are:

- **Read(A):** Read operations Read(A) or R(A) reads the value of A from the database and stores it in a buffer in the main memory.
- **Write (A):** Write operation Write(A) or W(A) writes the value back to the database from the buffer.

Let us take a debit transaction from an account that consists of the following operations:

1.R(A);
2.A=A-1000;
3.W(A);

Assume A's value before starting the transaction is 5000.

- The first operation reads the value of A from the database and stores it in a buffer.
- the Second operation will decrease its value by 1000. So buffer will contain 4000.
- the Third operation will write the value from the buffer to the database. So A's final value will be 4000.

But it may also be possible that the transaction may fail after executing some of its operations. The failure can be because of **hardware, software or power,** etc. For example, if the debit transaction discussed above fails after executing operation 2, the value of A will remain 5000 in the database which is not acceptable by the bank. To avoid this, Database has two important operations:

**Commit:** After all instructions of a transaction are successfully executed, the changes made by a transaction are made permanent in the database.

**Rollback:** If a transaction is not able to execute all operations successfully, all the changes made by a transaction are undone.

Concurrency Control in DBMS

- Executing a single transaction at a time will increase the waiting time of the other transactions which may result in delay in the overall execution. Hence for increasing the overall throughput and efficiency of the system, several transactions are executed.
- Concurrency control is a very important concept of DBMS which ensures the simultaneous execution or manipulation of data by several processes or user without resulting in data inconsistency.
- Concurrency control provides a procedure that is able to control concurrent execution of the operations in the database.

- The fundamental goal of database concurrency control is to ensure that concurrent execution of transactions does not result in a loss of database consistency. The concept of serializability can be used to achieve this goal, since all serializable schedules preserve consistency of the database. However, not all schedules that preserve consistency of the database are serializable.
- In general it is not possible to perform an automatic analysis of low-level operations by transactions and check their effect on database consistency constraints. However, there are simpler techniques. One is to use the database consistency constraints as the basis for a split of the database into subdatabases on which concurrency can be managed separately. Another is to treat some operations besides read and write as fundamental low-level operations and to extend concurrency control to deal with them.

## Problems with Concurrent Execution

In a database transaction, the two main operations are **READ** and **WRITE** operations. So, there is a need to manage these two operations in the concurrent execution of the transactions as if these operations are not performed in an interleaved manner, and the data may become inconsistent. So, the following problems occur with the Concurrent Execution of the operations:

### Problem 1: Lost Update Problems (W - W Conflict)

The problem occurs *when two different database transactions perform the read/write operations on the same database items in an interleaved manner (i.e., concurrent execution) that makes the values of the items incorrect hence making the database inconsistent.*

**For example:**

**Consider the below diagram where two transactions $T_X$ and $T_Y$, are performed on the same account A where the balance of account A is $300.**

| Time | $T_x$ | $T_y$ |
|---|---|---|
| $t_1$ | READ (A) | — |
| $t_2$ | A = A - 50 | |
| $t_3$ | — | READ (A) |
| $t_4$ | — | A = A + 100 |
| $t_5$ | — | — |
| $t_6$ | WRITE (A) | — |
| $t_7$ | | WRITE (A) |

**LOST UPDATE PROBLEM**

- At time t1, transaction $T_X$ reads the value of account A, i.e., $300 (only read).
- At time t2, transaction $T_X$ deducts $50 from account A that becomes $250 (only deducted and not updated/ write).

- Alternately, at time t3, transaction $T_Y$ reads the value of account A that will be $300 only because $T_X$ didn't update the value yet.
- At time t4, transaction $T_Y$ adds $100 to account A that becomes $400 (only added but not updated/write).
- At time t6, transaction $T_X$ writes the value of account A that will be updated as $250 only, as $T_Y$ didn't update the value yet.
- Similarly, at time t7, transaction $T_Y$ writes the values of account A, so it will write as done at time t4 that will be $400. It means the value written by $T_X$ is lost, i.e., $250 is lost.

Hence data becomes incorrect, and database sets to inconsistent.

## Dirty Read Problems (W-R Conflict)

The dirty read problem occurs *when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction. There comes the Read-Write Conflict between both transactions.*

**For example:**

**Consider two transactions $T_X$ and $T_Y$ in the below diagram performing read/write operations on account A where the available balance in account A is $300:**

| Time | $T_x$ | $T_y$ |
|---|---|---|
| $t_1$ | READ (A) | — |
| $t_2$ | A = A + 50 | — |
| $t_3$ | WRITE (A) | — |
| $t_4$ | — | READ (A) |
| $t_5$ | SERVER DOWN ROLLBACK | — |

**DIRTY READ PROBLEM**

- At time t1, transaction $T_X$ reads the value of account A, i.e., $300.
- At time t2, transaction $T_X$ adds $50 to account A that becomes $350.
- At time t3, transaction $T_X$ writes the updated value in account A, i.e., $350.
- Then at time t4, transaction $T_Y$ reads account A that will be read as $350.
- Then at time t5, transaction $T_X$ rollbacks due to server problem, and the value changes back to $300 (as initially).
- But the value for account A remains $350 for transaction $T_Y$ as committed, which is the dirty read and therefore known as the Dirty Read Problem.

## Unrepeatable Read Problem (W-R Conflict)

*Also known as Inconsistent Retrievals Problem that occurs when in a transaction, two different values are read for the same database item.*

**For example:**

**Consider two transactions, $T_X$ and $T_Y$, performing the read/write operations on account A, having an available balance = $300. The diagram is shown below:**

| Time | $T_x$ | $T_y$ |
|---|---|---|
| $t_1$ | READ (A) | — |
| $t_2$ | — | READ (A) |
| $t_3$ | — | A = A + 100 |
| $t_4$ | — | WRITE (A) |
| $t_5$ | READ (A) | — |

**UNREPEATABLE READ PROBLEM**

- At time t1, transaction $T_X$ reads the value from account A, i.e., $300.
- At time t2, transaction $T_Y$ reads the value from account A, i.e., $300.
- At time t3, transaction $T_Y$ updates the value of account A by adding $100 to the available balance, and then it becomes $400.
- At time t4, transaction $T_Y$ writes the updated value, i.e., $400.
- After that, at time t5, transaction $T_X$ reads the available value of account A, and that will be read as $400.
- It means that within the same transaction $T_X$, it reads two different values of account A, i.e., $ 300 initially, and after updation made by transaction $T_Y$, it reads $400. It is an unrepeatable read and is therefore known as the Unrepeatable read problem.

Thus, in order to maintain consistency in the database and avoid such problems that take place in concurrent execution, management is needed, and that is where the concept of Concurrency Control comes into role.

# Concurrency Control

Concurrency Control is the working concept that is required for controlling and managing the concurrent execution of database operations and thus avoiding the inconsistencies in the database. Thus, for maintaining the concurrency of the database, we have the concurrency control protocols.

## Concurrency Control Protocols

The concurrency control protocols ensure the *atomicity, consistency, isolation, durability* and *serializability* of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:

- Lock Based Concurrency Control Protocol
- Time Stamp Concurrency Control Protocol
- Validation Based Concurrency Control Protocol

## 3.10 Two Phase Locking:

The Two-Phase Locking (2PL) Protocol is an essential concept in database management systems used to maintain data consistency and ensure smooth operation when multiple transactions are happening simultaneously. It helps to prevent issues like data conflicts where two or more transactions try to access or modify the same data at the same time, potentially causing errors.

Two-Phase Locking is widely used to ensure serializability, meaning transactions occur in a sequence that maintains data accuracy. This article will explore the workings of the 2PL protocol, its types, advantages and its role in maintaining a reliable database system
Two Phase Locking

The Two-Phase Locking (2PL) Protocol is a key technique used in database management systems to manage how multiple transactions access and modify data at the same time. When many users or processes interact with a database, it's important to ensure that data remains consistent and error-free. Without proper management, issues like data conflicts or corruption can occur if two transactions try to use the same data simultaneously.

The Two-Phase Locking Protocol resolves this issue by defining clear rules for managing data locks. It divides a transaction into two phases:

1. **Growing Phase**: In this step, the transaction gathers all the locks it needs to access the required data. During this phase, it cannot release any locks.
2. **Shrinking Phase**: Once a transaction starts releasing locks, it cannot acquire any new ones. This ensures that no other transaction interferes with the ongoing process.

**Types of Lock**

**i) Shared Lock (S):** Shared Lock is also called a read-only lock, allows multiple transactions to access the same data item for reading at the same time. However, transactions with this lock cannot make changes to the data. A shared lock is requested using the lock-S instruction.

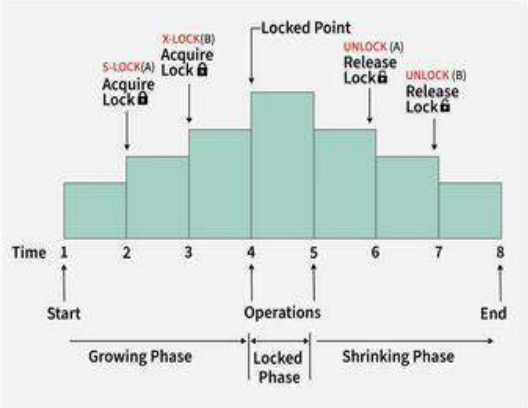
**ii) Exclusive Lock (X):** An Exclusive Lock allows a transaction to both read and modify a data item. This lock is exclusive, meaning no other transaction can access the same data item while this lock is held. An exclusive lock is requested using the lock-X instruction.

**Lock Conversions**

In the Two-Phase Locking Protocol, lock conversion means changing the type of lock on data while a transaction is happening. This process is carefully controlled to maintain consistency in the database.

1. **Upgrading a Lock**: This means changing a shared lock (S) to an exclusive lock (X). For example, if a transaction initially only needs to read data (S) but later decides it needs to update the same data, it can request an upgrade to an exclusive lock (X). However, this can only happen during the Growing Phase, where the transaction is still acquiring locks.
    - Example: A transaction reads a value (S lock) but then realizes it needs to modify the value. It upgrades to an X lock during the Growing Phase.
2. **Downgrading a Lock**: This means changing an exclusive lock (X) to a shared lock (S). For instance, if a transaction initially planned to modify data (X lock) but later decides it only needs to read it, it can downgrade the lock. However, this must happen during the Shrinking Phase, where the transaction is releasing locks.

- Example: A transaction modifies a value (X lock) but later only needs to read the value, so it downgrades to an S lock during the Shrinking Phase.




*2PL-Locking*

These rules ensure that the Two-Phase Locking Protocol maintains consistency and avoids conflicts between transactions. By limiting when upgrades and downgrades can occur, the system prevents situations where multiple transactions interfere with each other's operations.

Let's see a transaction implementing 2-PL.

| | T1 | T2 |
|---|---|---|
| 1 | lock-S(A) | |
| 2 | | lock-S(A) |
| 3 | lock-X(B) | |
| 4 | ..... | ..... |
| 5 | Unlock(A) | |
| 6 | | Lock-X(C) |
| 7 | Unlock(B) | |
| 8 | | Unlock(A) |
| 9 | | Unlock(C) |
| 10 | ..... | ..... |

This is a basic outline of a transaction that demonstrates how locking and unlocking work in the Two-Phase Locking Protocol (2PL).

### Transaction $T_1$

- The growing Phase is from steps 1-3
- The shrinking Phase is from steps 5-7
- Lock Point at 3

### Transaction $T_2$

- The growing Phase is from steps 2-6
- The shrinking Phase is from steps 8-9
- Lock Point at 6

## Lock Point

The lock point in a transaction is the moment when the transaction finishes acquiring all the locks it needs. After this point, no new locks can be added, and the transaction starts releasing locks. It's a key step in the Two-Phase Locking Protocol to ensure the rules of growing and shrinking phases are followed.

## Example of 2PL

Imagine a library system where multiple users can borrow or return books. Each action (like borrowing or returning) is treated as a transaction. Here's how the Two-Phase Locking Protocol (2PL) works, including the lock point:

User A wants to:

1. Check the availability of Book X.
2. Borrow Book X if it's available.
3. Update the library's record.

**Growing Phase (Locks are Acquired):**

1. User A locks Book X with a shared lock (S) to check its availability.
2. After confirming the book is available, User A upgrades the lock to an exclusive lock (X) to borrow it.
3. User A locks the library's record to update the borrowing details.

**Lock Point:** Once User A has acquired all the necessary locks (on Book X and the library record), the transaction reaches the lock point. No more locks can be acquired after this.

**Shrinking Phase (Locks are Released):**

1. User A updates the record and releases the lock on the library's record.
2. User A finishes borrowing and releases the exclusive lock on Book X.

This process ensures that no other user can interfere with Book X or the library record during the transaction, maintaining data accuracy and consistency. The lock point ensures that all locks are acquired before any are released, following the 2PL rules.

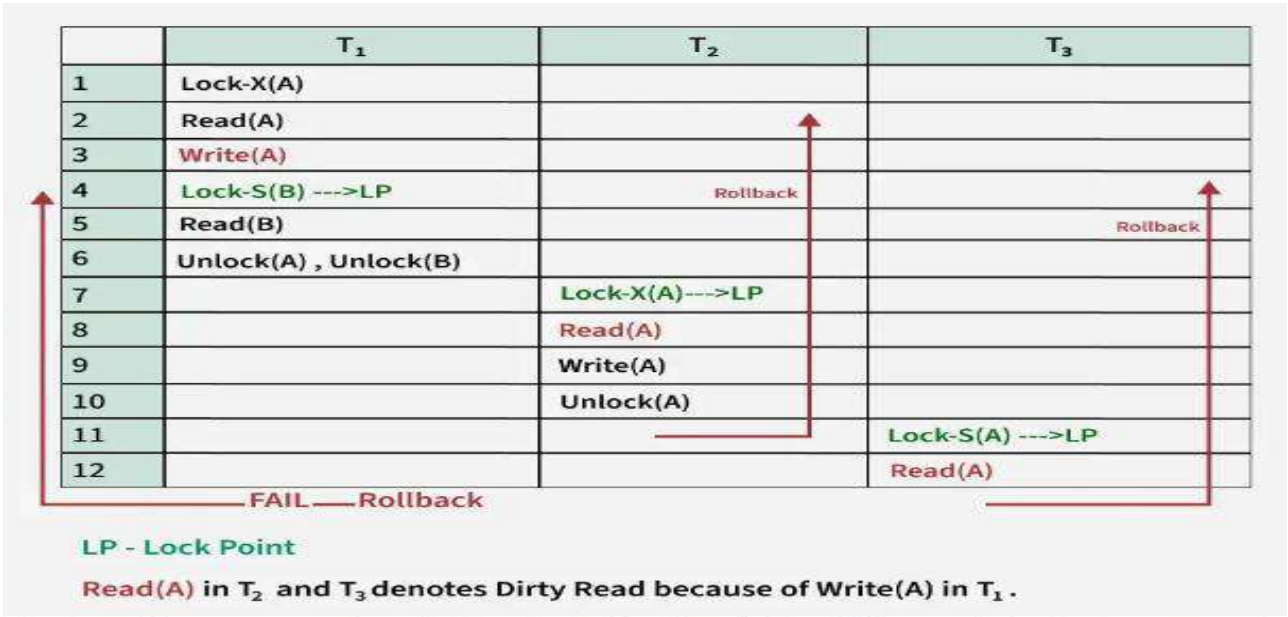
## Drawbacks of 2PL

Two-phase locking (2PL) ensures that transactions are executed in the correct order by using two phases: acquiring and releasing locks. However, it has some drawbacks:

- **Deadlocks**: Transactions can get stuck waiting for each other's locks, causing them to freeze indefinitely.
- **Cascading Rollbacks**: If one transaction fails, others that depend on it might also fail leading to inefficiency and potential data issues.
- **Lock Contention**: Too many transactions competing for the same locks can slow down the system, especially when many users are working at the same time.
- **Limited Concurrency**: The strict rules of 2PL can reduce how many transactions can run at once, resulting in slower performance and longer wait times.

### Cascading Rollbacks in 2-PL

Let's see the following Schedule:




The image illustrates a transaction schedule using the Two-Phase Locking (2PL) protocol, showing the sequence of actions for three transactions T1, T2 and T3.

**Key Points:**

1. **Transaction T1:**
   - T1 acquires an exclusive lock (X) on data item A, performs a write operation on A and then acquires a shared lock (S) on B.
   - T1 reaches its lock point (LP) after acquiring all locks.
   - Eventually, T1 fails and a rollback is triggered, undoing its changes.

2. **Transaction T2:**
   - T2 reads A after T1 writes A. This is called a dirty read because T1's write is not committed yet.
   - When T1 rolls back, T2's operations become invalid, and it is also forced to rollback.

3. **Transaction T3:**
   - T3 reads A after T2 reads A. Since T2 depends on the uncommitted changes of T1, T3 indirectly relies on T1's changes.
   - When T1 rolls back, T3 is also forced to rollback even though it was not directly interacting with T1's operations.

**Cascading Rollback Problem:**

- Here, T2 and T3 both depend on uncommitted data from T1 (either directly or indirectly).
- When T1 fails and rolls back all dependent transactions (T2 and T3) must also rollback because their operations were based on invalid data.
- Cascading rollbacks waste system resources, reduce concurrency and lead to inefficiency.
- In large systems, this can significantly affect performance and make recovery more complex.

  **How to avoid it?**

  - **Strict 2PL**: Ensure that transactions release their locks only after they commit, preventing other transactions from accessing uncommitted changes.
  - **Rigorous 2PL**: Extend strict 2PL to hold both read and write locks until the transaction commits, offering even stronger guarantees.

### Deadlock in 2-PL

Consider this simple example. We have two transactions $T_1$ and $T_2$.

```
Schedule:  Lock-X1(A)  Lock-X2(B)  Lock-X1(B)  Lock-X2(A)
```

This sequence represents a locking scenario where two transactions, T1 and T2 are trying to lock two resources, A and B in a particular order. Here's what each step means:

1. **Lock-X1(A):**
   Transaction T1 acquires an exclusive lock on resource A. This means T1 has full control over A and no other transaction can use it until T1 releases the lock.
2. **Lock-X2(B):**
   Transaction T2 acquires an exclusive lock on resource B. Similarly, T2 now has full control over B and no other transaction can access B until T2 releases the lock.
3. **Lock-X1(B):**
   Transaction T1 tries to acquire an exclusive lock on resource B but T2 already holds the lock on B. So, T1 must wait for T2 to release the lock.
4. **Lock-X2(A):**
   At the same time, Transaction T2 tries to acquire an exclusive lock on resource A but T1 already holds the lock on A. So, T2 must wait for T1 to release the lock.

The above-mentioned type of 2-PL is called Basic 2PL. To sum it up, it ensures Conflict Serializability but does not prevent Cascading Rollback and Deadlock. To prevent from these issues Strict Two Phase Locking and Rigorous Two Phase Locking can be used.

## Categories of Two-Phase Locking (Strict, Rigorous & Conservative)

Two-Phase Locking (2PL) is a crucial technique in database management systems designed to ensure consistency and isolation during concurrent transactions. This article will cover the three main types of 2PL: strict 2PL, rigorous 2PL and conservative 2PL highlighting the differences in their locking protocols

There are three categories:

1. Strict 2-PL
2. Rigorous 2-PL
3. Conservative 2-PL

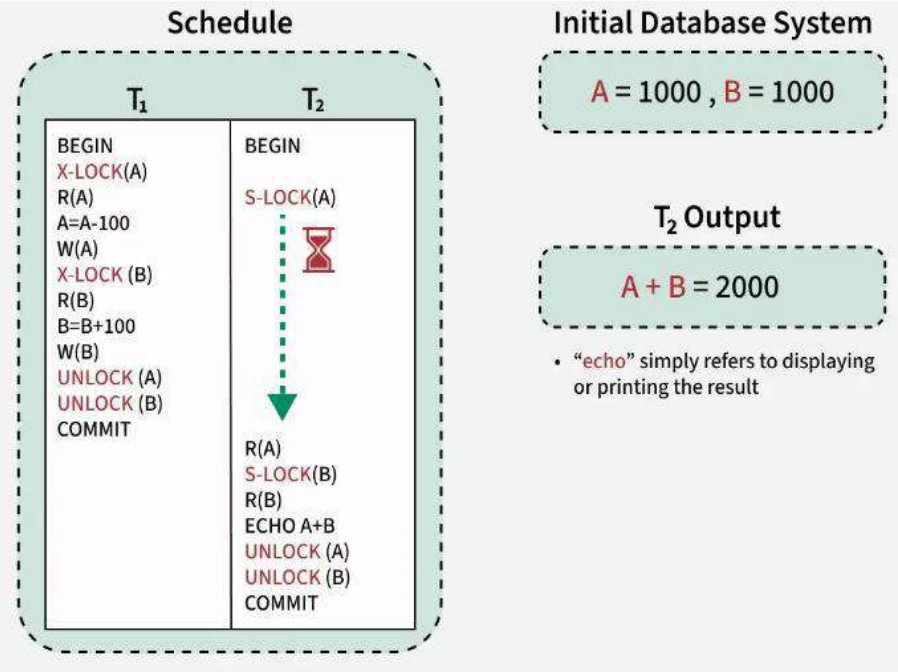
## Strict Two-Phase Locking Protocol (Strict 2PL)

The Strict Two-Phase Locking Protocol is a variation of the Two-Phase Locking (2PL) protocol that adds a specific rule for releasing exclusive (X) locks. In addition to following the two phases (growing and shrinking), strict 2PL ensures that:

1. **All Exclusive (X) Locks are Held Until the Transaction Commits or Aborts:**
   - A transaction can acquire locks during the growing phase.
   - It cannot release any exclusive locks until it has completed its operations and either committed or aborted.
   - This rule prevents other transactions from accessing data modified by an ongoing transaction, ensuring data consistency.

2.**Benefits:**

- **Recoverable Schedule:** Ensures that if a transaction fails, no other transactions depend on its uncommitted changes.
- **Cascadeless Schedule:** Prevents cascading rollbacks, as other transactions cannot read uncommitted data.




The image demonstrates a Strict Two-Phase Locking (2PL) Protocol scenario with two transactions, T1 and T2 accessing and modifying two database accounts A and B. Here's a step-by-step explanation:

Initial Database State:

- Account A=1000
- Account B=1000

Steps in the Schedule:

***Transaction T1:***

1.BEGIN**:** T1 starts and acquires an exclusive lock (X-Lock) on A. This means T1 can read and modify A, and no other transaction can access A during this time.
2.T1 reads A and subtracts 100 from it (A=900).
3.T1 acquires an X-Lock on B and adds 100 to B (B=1100).
4.T1 unlocks A and B only after committing. At this point, the changes made by T1 become visible to other transactions.

***Transaction T2:***

1.BEGIN**:** T2 starts and tries to acquire a shared lock (S-Lock) on A to read its value. 2.T2 waits because T1 is holding an exclusive lock on A. This prevents T2 from reading the uncommitted data ensuring no dirty reads.
3.Once T1 commits and releases the lock on A, T2 acquires the S-Lock on A, reads A=900 and moves on.
4.T2 acquires an S-Lock on B, reads B=1100 and calculates A+B=2000.
5.T2 unlocks both A and B after committing.

**Final Output:**

- After T1 commits, T2 reads the updated values of A and B and the result A+B=2000 reflects the committed changes.

**Rigorous Two-Phase Locking (Rigorous 2PL)**

Rigorous 2PL is a stricter version of the Two-Phase Locking (2PL) protocol. In this protocol:

1. All locks (both shared and exclusive) are held by a transaction until it either commits or aborts.
2. Locks are only released after the transaction finishes, ensuring that no other transaction can access the locked data until the first transaction is fully complete.

This means that no other transaction can read or write the data being used by the current transaction until it is committed or rolled back. This ensures data consistency and avoids issues like dirty reads or cascading rollbacks.

**Conservative Two-Phase Locking (Conservative 2PL)**

Conservative 2PL is a variation of the Two-Phase Locking protocol designed to prevent deadlocks entirely. It is also known as the Static-2PL. The key feature of Conservative 2PL is that a transaction acquires all the locks it needs at the very beginning of the transaction before it starts executing. If the transaction cannot acquire all the required locks, it does not proceed and waits until all locks become available.

| $T_1$ | $T_2$ |
|---|---|
| lock-exclusive(a) | |
| lock-exclusive(b) | |
| read(a) | |
| a=a+50 | |
| write(a) | |
| unlock(a) | |
| | lock-exclusive(a) |
| | read(a) |
| | a=a+50 |
| | write(a) |
| | unlock(a) |
| read(b) | |
| b=b+100 | |
| write(b) | |
| unlock(b) | |

This example demonstrates Conservative Two-Phase Locking (Conservative 2PL), where each transaction acquires all the locks it needs at the beginning of the transaction to avoid deadlocks. Here's a detailed explanation of how it aligns with the principles of Conservative 2PL:

***Transaction T1:***

1. **Lock-Exclusive(A):**
   - T1 acquires an exclusive lock on A before performing any operation on it.
2. **Lock-Exclusive(B):**
   - Before proceeding further, T1 also acquires an exclusive lock on B ensuring that both resources required by T1 are locked at the start.
3. **Operations on A:**
   - T1 reads the value of A, increments it by 50 and writes the updated value back to A.
4. **Unlock(A):**
   - After completing all operations on A, T1 releases the lock on A.
5. **Operations on B:**
   - T1 reads the value of B, increments it by 100 and writes the updated value back to B.
6. **Unlock(B):**
   - T1 releases the lock on B after finishing all operations.

***Transaction T2:***

1. **Lock-Exclusive(A):**
   - T2 attempts to acquire an exclusive lock on A at the start of the transaction. If T1 still holds the lock on A T2 waits until T1 releases it.
2. **Operations on A:**
   - Once T2 acquires the lock it reads A, increments it by 50 and writes the updated value back to A.
3. **Unlock(A):**
   - T2 releases the lock on A, after completing its operations.

**Advantages of Conservative 2PL:**

1. **Deadlock-Free:**
   - Deadlocks are avoided because a transaction either acquires all required locks at once or waits until it can.
2. **Efficient Use of Locks:**
   - Transactions do not hold partial locks while waiting for others, reducing contention.
3. **Consistency and Serializability:**
   - Like all 2PL variants, it ensures consistent and serializable schedules.

| S.No. | Conservative 2-PL | Strict 2-PL |
|---|---|---|
| 1. | In Conservative 2-PL, A transaction has to acquire locks on all the data items it requires before the transaction begins it execution. | In Strict 2-PL, A transaction can acquire locks on data items whenever it requires (only in growing phase) during its execution. |
| 2. | It does not have growing phase. | It has growing phase. |
| 3. | It has shrinking phase. | It has partial shrinking phase. |
| 4. | It ensures that the schedule generated would be Serializable and Deadlock-Free. | It ensures that the schedule generated would be Serializable, Recoverable and Cascadeless. |
| 5. | It does not ensures Recoverable and Cascadeless schedule. | It does not ensures Deadlock-Free schedule. |
| 6. | It does not ensure Strict Schedule. | It ensures that the schedule generated would be Strict. |
| 7. | It is less popular as compared to Strict 2-PL. | It is the most popular variation of 2-PL. |
| 8. | It is not used in practise. | It is the most popular variation of 2-PL. |
| 9. | In Conservative 2-PL, a transaction can read a value of uncommitted transaction. | In Strict 2-PL, a transaction only reads value of committed transaction. |

| S.No. | Conservative 2-PL | Rigorous 2-PL |
|---|---|---|
| 1. | A transaction has to acquire locks on all the data items it requires before the transaction begins it execution. | A transaction can acquire locks on data items whenever it requires (only in growing phase) during its execution. |
| 2. | It does not have a growing phase. | It has a growing phase. |
| 3. | It has a shrinking phase. | It does not have a shrinking phase. |
| 4. | It ensures that the schedule generated would be Serializable and Deadlock-Free. | It ensures that the schedule generated would be Serializable, Recoverable and Cascadeless. |
| 5. | It does not ensures Recoverable and Cascadeless schedule. | It does not ensures Deadlock-Free schedule. |
| 6. | It does not ensure Strict Schedule. | It ensures that the schedule generated would be Strict. |
| 7. | It is not used in practise as its difficult to implement. | It is easy to implement but a lighter version of it (i.e Strict 2-PL) is used in practise. |
| 8. | In Conservative 2-PL, a transaction can read a value of uncommitted transaction. | In Rigorous 2-PL, a transaction only reads value of committed transaction. |

## 3.11 Timestamp:

### Timestamp Ordering Protocol

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.
- The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.
- Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.
- The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

**Basic Timestamp ordering protocol works as follows:**

The basic timestamp ordering methods make sure that any conflicting read and write operations are executed.

1. Check the following condition whenever a transaction Ti issues a **Read (X)** operation:

- If W_TS(X) >TS(Ti) then the operation is rejected.
- If W_TS(X) <= TS(Ti) then the operation is executed.
- Timestamps of all the data items are updated.

2. Check the following condition whenever a transaction Ti issues a **Write(X)** operation:

- If TS(Ti) < R_TS(X) then the operation is rejected.
- If TS(Ti) < W_TS(X) then the operation is rejected and Ti is rolled back otherwise the operation is executed.

**Where,**

**TS(TI)** denotes the timestamp of the transaction Ti.

**R_TS(X)** denotes the Read time-stamp of data-item X.

**W_TS(X)** denotes the Write time-stamp of data-item X.

### Advantages and Disadvantages of TO protocol:

- TO protocol ensures serializability since the precedence graph is as follows:




**Image:** Precedence Graph for TS ordering

- TS protocol ensures freedom from deadlock that means no transaction ever waits.
- But the schedule may not be recoverable and may not even be cascade- free.

**To illustrate this mechanism, we shall consider two transactions T1 and T2. Both the transaction subtracts a 200 from A and adds it to B. The timestamps of T1 and T2 are 5:29 pm and 5:30 pm respectively. Initially all the system clock is set to zero.**

Once restarted the transaction T1 will acquire new timestamp (say 5:31 pm).

| Schedule | Read_TS(A) | Write_TS(A) | Read_TS(B) | Write_TS(B) | Description |
|---|---|---|---|---|---|
| T1: Read (A) | 5 : 29 | 0 | 0 | 0 | After T1 reads the Read_TS (A) becomes 5 : 29 and remains so till T1 reads B. |
| T1: A: = A -200 | 5 : 29 | 0 | 0 | 0 | |
| T1: Read (B) | 5 : 29 | 0 | 5 : 29 | 0 | Now both Read_TS (A), Read_TS(B) becomes 5 : 29 |
| T2: Read (A) | 5 : 30 | 0 | 5 : 29 | 0 | Using rule 1(b) since 5: 30 > 5:29 so Read_TS(A) is updated. |
| T2: Read (B) | 5 : 30 | 0 | 5 : 30 | 0 | Using rule 1(b) since 5: 30 > 5:29 so Read_TS(B) is updated. |
| T2: A: = A -200 | - | - | - | | |
| T2: B: = B + 200 | - | - | - | | No read or write operation. |
| T2: Write (A) | 5 : 30 | 5 : 30 | 5 : 30 | 0 | Using rule 2(c), Write_TS (A) updated. |
| T2: Write (B) | 5 : 30 | 5 : 30 | 5 : 30 | 5 : 30 | Using rule 2(c), Write_TS (B) updated. |
| T1: B: = B + 200 | - | - | | | No read or write operation. |
| T1: Write (A) | 5 : 30 | conflict | 5 : 30 | 5 : 30 | Using rule 2(b), since 5: 30 > 5:29 so transaction T1 is aborted. |
| T1: Write (B) | | | | | |

It can be verified that schedule produces the correct result even if T1 has aborted but this is not always the case. This method guarantees that the transactions are conflict serializable and the results are equivalent to a serial schedule in which transaction are executed in the timestamp order. The basic timestamp ordering method would be the same as if all transactions were executed one after the other without any interference.

**Timestamp based Concurrency Control**

Timestamp-based concurrency control is a method used in database systems to ensure that transactions are executed safely and consistently without conflicts, even when multiple transactions are being processed simultaneously. This approach relies on timestamps to manage and coordinate the execution order of transactions. Refer to the timestamp of a transaction *T* as ***TS(T)***.

**What is Timestamp Ordering Protocol?**

The main idea for this protocol is to order the transactions based on their Timestamps. A schedule in which the transactions participate is then serializable and the only *equivalent serial schedule permitted* has the transactions in the order of their Timestamp Values. Stating simply, the schedule is equivalent to the particular *Serial Order* corresponding to the *order of the Transaction timestamps*. An algorithm must ensure that, for each item accessed by *Conflicting Operations* in the schedule, the order in which the item is accessed does not violate the ordering. To ensure this, use two Timestamp Values relating to each database item **X**.

- **W_TS(X)** is the largest timestamp of any transaction that executed **write(X)** successfully.
- **R_TS(X)** is the largest timestamp of any transaction that executed **read(X)** successfully.

**Basic Timestamp Ordering**

Every transaction is issued a timestamp based on when it enters the system. Suppose, if an old transaction $T_i$ has timestamp $TS(T_i)$, a new transaction $T_j$ is assigned timestamp $TS(T_j)$ such that **$TS(T_i) < TS(T_j)$**. The protocol manages concurrent execution such that the timestamps determine the serializability order. The timestamp ordering protocol ensures that any conflicting read and write operations are executed in timestamp order. Whenever some Transaction *T* tries to issue a R_item(X) or a W_item(X), the Basic TO algorithm compares the timestamp of *T* with **R_TS(X) & W_TS(X)** to ensure that the Timestamp order is not violated. This describes the Basic TO protocol in the following two cases.

- Whenever a Transaction *T* issues a **W_item(X)** operation, check the following conditions:
  - If ***R_TS(X) > TS(T)*** and if ***W_TS(X) > TS(T)***, then abort and rollback T and reject the operation. else,
  - Execute W_item(X) operation of T and set W_TS(X) to TS(T).
- Whenever a Transaction *T* issues a **R_item(X)** operation, check the following conditions:
  - If ***W_TS(X) > TS(T)***, then abort and reject T and reject the operation, else
  - If W_TS(X) <= TS(T), then execute the R_item(X) operation of T and set R_TS(X) to the larger of TS(T) and current R_TS(X).

Whenever the Basic TO algorithm detects two conflicting operations that occur in an incorrect order, it rejects the latter of the two operations by aborting the Transaction that issued it. Schedules produced by Basic TO are guaranteed to be *conflict serializable*. Already discussed that using Timestamp can ensure that our schedule will be *deadlock free*. One drawback of the Basic TO protocol is that **Cascading Rollback** is still possible. Suppose we have a Transaction $T_1$ and $T_2$ has used a value written by $T_1$. If $T_1$ is aborted and resubmitted to the system then, $T_2$ must also be aborted and rolled back. So the problem of Cascading aborts still prevails. Let's gist the Advantages and Disadvantages of Basic TO protocol:

- Timestamp Ordering protocol ensures serializability since the precedence graph will be of the form:
- Timestamp protocol ensures freedom from deadlock as no transaction ever waits.

**Strict Timestamp Ordering**

A variation of Basic TO is called **Strict TO** ensures that the schedules are both Strict and Conflict Serializable. In this variation, a Transaction T that issues a R_item(X) or W_item(X) such that TS(T) > W_TS(X) has its read or write operation delayed until the Transaction **T'** that wrote the values of X has committed or aborted.

**Advantages**

- **High Concurrency:** Timestamp-based concurrency control allows for a high degree of concurrency by ensuring that transactions do not interfere with each other.
- **Efficient:** The technique is efficient and scalable, as it does not require locking and can handle a large number of transactions.
- **No Deadlocks:** Since there are no locks involved, there is no possibility of deadlocks occurring.
- **Improved Performance:** By allowing transactions to execute concurrently, the overall performance of the database system can be improved.

**Disadvantages**

- **Limited Granularity:** The granularity of timestamp-based concurrency control is limited to the precision of the timestamp. This can lead to situations where transactions are unnecessarily blocked, even if they do not conflict with each other.
- **Timestamp Ordering:** In order to ensure that transactions are executed in the correct order, the timestamps need to be carefully managed. If not managed properly, it can lead to inconsistencies in the database.
- **Timestamp Synchronization:** Timestamp-based concurrency control requires that all transactions have synchronized clocks. If the clocks are not synchronized, it can lead to incorrect ordering of transactions.
- **Timestamp Allocation:** Allocating unique timestamps for each transaction can be challenging, especially in distributed systems where transactions may be initiated at different locations.

## 3.12 Multi-version:

Multi-Version Concurrency Control (MVCC) is a database optimization method, that makes redundant copies of records to allow for safe concurrent reading and updating of data. DBMS reads and writes are not blocked by one another while using MVCC. A technique called concurrency control keeps concurrent processes running to avoid read/write conflicts or other irregularities in a database. Whenever it has to be updated rather than replacing the old one with the new information an MVCC database generates a newer version of the data item.

**What is Multi-Version Concurrency Control (MVCC) in DBMS?**

Multi-Version Concurrency Control is a technology, utilized to enhance databases by resolving concurrency problems and also data locking by preserving older database versions. When many tasks attempt to update the same piece of data simultaneously, MVCC causes a conflict and necessitates a retry from one or more of the processes.

**Types of Multi-Version Concurrency Control (MVCC) in DBMS**

- **Timestamp-based MVCC:** The data visibility to transactions is defined by the unique timestamp assigned to each transaction that creates a new version of a record.
- **Snapshot-based MVCC:** This utilizes the database snapshot that is created at the beginning of a transaction to supply the information that is needed for the transaction.
- **History-based MVCC**: This Keeps track of every modification made to a record, making transaction rollbacks simple.

- **Hybrid MVCC: This** coordinates data flexibility and performance by combining two or more MVCC approaches.

**How Does Multi-Version Concurrency Control (MVCC) in DBMS Works?**

- In the database, every tuple has a version number. The tuple with the greatest version number can have a read operation done on it simultaneously.
- Only a copy of the record may be used for writing operations.
- While the copy is being updated concurrently, the user may still view the previous version.
- The version number is increased upon successful completion of the writing process.
- The upgraded version is now used for every new record operation and every time there is an update, this cycle is repeated.

**Implementation of Multi-Version Concurrency Control (MVCC) in DBMS**

- MVCC operates time stamps (TS) and increases transaction IDs to assure transactional consistency. MVCC manage many copies of the object, ensuring that a transaction (T) never has to wait to read a database object (P).
- A specific transaction Ti can read the most recent version of the object, which comes before the transaction's Read Timestamp RTS(Ti) since each version of object P contains both a Read Timestamp and a Write Timestamp.
- If there are other pending transactions to the same object that have an earlier Read Timestamp (RTS), then a Write cannot be completed.
- You cannot finish your checkout transaction until the person in front of you has finished theirs, much as when you are waiting in a queue at the shop.
- To reiterate, each object (P) has a Timestamp (TS). Should transaction Ti attempt to Write to an object and its Timestamp (TS) exceeds the object's current Read Timestamp, TS(Ti)<RTS(P)TS(Ti)<RTS(P), the transaction will be cancelled and retried.
- Ti makes a new copy of object P and sets its read/write timestamp (TS) to the transaction timestamp (TS↞TS(Ti))(TS↞TS(Ti)).

**Advantages of Multi-Version Concurrency Control (MVCC) in DBMS**

- **The reduced read-and-writenecessity for database locks:** The database can support many read-and-write transactions without locking the entire system thanks to MVCC.
- **Increased Concurrency:** This Enables several users to use the system at once.
- **Minimize read operation delays:** By enabling concurrent read operations, MVCC helps to cut down on read times for data.
- **Accuracy and consistency:** Preserves data integrity over several concurrent transactions.

**Disadvantages of Multi-Version Concurrency Control (MVCC) in DBMS**

- **Overhead:** Keeping track of many data versions might result in overhead.
- **Garbage collecting:** To get rid of outdated data versions, MVCC needs effective garbage collecting systems.
- **Increase the size of the database:** Expand the capacity of the database since MVCC generates numerous copies of the records and/or tuples.
- **Complexity:** Compared to more straightforward locking systems, MVCC usage might be more difficult.

**Multi-version concurrency control (MVCC)**

Multi-version protocol aims to reduce the delay for read operations. It maintains multiple versions of data items. Whenever a write operation is performed, the protocol creates a new version of the transaction data to ensure conflict-free and successful read operations.

The newly created version contains the following information

- **Content** − This field contains the data value of that version.
- **Write_timestamp** − This field contains the timestamp of the transaction that created the new version.
- **Read_timestamp** − This field contains the timestamp of the transaction that will read the newly created value.

By creating multiple versions of the data, the multi-version protocol ensures that read operations can access the appropriate version of the data without encountering conflicts. The protocol thus enables efficient concurrency control and reduces delays in read operations.

## Various Types of MVCC

These are compared in the following table

| MVCC Type | Description | Advantages | Disadvantages |
|---|---|---|---|
| Snapshot-based | Creates a snapshot of the database at the start of a transaction and uses it to provide necessary data for the transaction | Easy to implement | Significant overhead due to storing multiple versions of data |
| Timestamp-based | Assigns a unique timestamp to each transaction that creates a new version of a record; used to determine data visibility to transactions | More efficient than snapshot-based MVCC | Requires additional storage to store timestamps |
| History-based | Stores a complete history of all changes made to a record, allowing for easy rollback of transactions | Provides highest level of data consistency | Most complex of the MVCC techniques |
| Hybrid | Combines two or more MVCC techniques to balance performance and data consistency | Provides benefits of multiple MVCC techniques | More complex to implement than individual techniques |

**Benefits of multi-version concurrency control (MVCC)**

MVCC is a technique that helps databases manage multiple transactions happening at the same time. When a DBMS implements MVCC correctly, it provides several benefits, such as − i) Less need for database locks

With MVCC, the database can allow multiple transactions to read and write data without locking the entire database.

ii) Fewer issues with multiple transactions trying to access the same data

MVCC helps reduce conflicts between transactions accessing the same data.

iii) Faster access to read data

Since MVCC allows multiple transactions to read data at the same time, it improves the speed of reading data.

iv) Records are still protected during write operations

MVCC ensures that data is protected from being changed by other transactions while a transaction is making changes to it.

v) Fewer database deadlocks

Deadlocks occur when two or more transactions are waiting for each other to release a lock, causing the system to come to a halt. MVCC can reduce the number of these occurrences.

**Drawbacks of Multi-version concurrency control (MVCC)**

Although multi-version concurrency control (MVCC) provides several benefits, there are two main disadvantages to this approach

- Concurrent update control methods can be challenging to implement.
- The database can become bloated with multiple versions of records, which increases its overall size.

## 3.13 Multiple Granularity Locking:

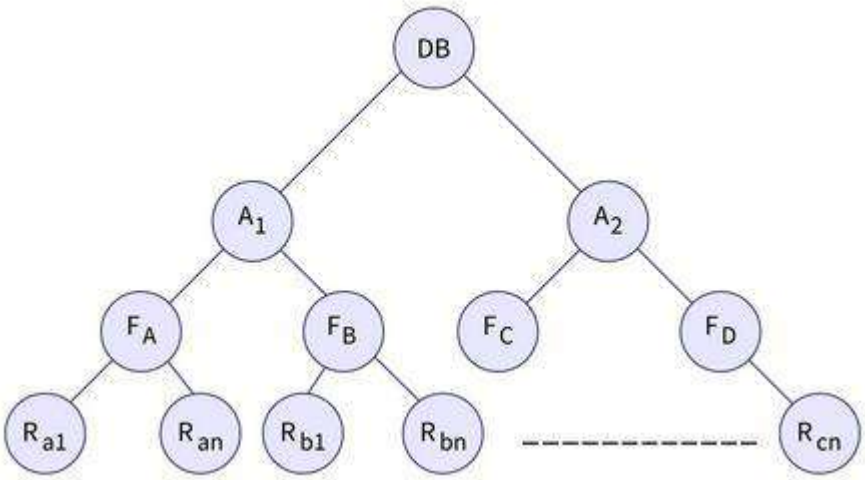
The various Concurrency Control schemes have used different methods and every individual Data item is the unit on which synchronization is performed. A certain drawback of this technique is if a transaction $T_i$ needs to access the entire database, and a locking protocol is used, then $T_i$ must lock each item in the database. It is less efficient, it would be simpler if $T_i$ could use a single lock to lock the entire database. But, if it considers the second proposal, this should not in fact overlook certain flaws in the proposed method. Suppose another transaction just needs to access a few data items from a database, so locking the entire database seems to be unnecessary moreover it may cost us a loss of Concurrency, which was our primary goal in the first place. To bargain between Efficiency and Concurrency use Granularity.

**Granularity**

It is the size of the data item allowed to lock. Now *Multiple Granularity* means hierarchically breaking up the database into blocks that can be locked and can be tracked what needs to lock and in what fashion. Such a hierarchy can be represented graphically as a tree. **For example,** consider the tree, which consists of four levels of nodes. The highest level represents the entire database. Below it is nodes of type area; the database consists of exactly these areas. The area has children nodes which are called files. Every area has those files that are its child nodes. No file can span more than one area.

Finally, each file has child nodes called records. As before, the file consists of exactly those records that are its child nodes, and no record can be present in more than one file. Hence, the levels starting from the top level are:

- database
- area
- file
- record




Consider the above diagram for the example given, each node in the tree can be locked individually. As in the 2-phase locking protocol, it shall use shared and exclusive lock modes. When a transaction locks a node, in either shared or exclusive mode, the transaction also implicitly locks all the descendants of that node in the same lock mode. For example, if transaction $T_i$ gets an explicit lock on file $F_c$ in exclusive mode, then it has an implicit lock in exclusive mode on all the records belonging to that file. It does not need to lock the individual records of $F_c$ explicitly. this is the main difference between Tree-Based Locking and Hierarchical locking for multiple granularities.
Now, with locks on files and records made simple, how does the system determine if the root node can be locked? One possibility is for it to search the entire tree but the solution nullifies the whole purpose of the multiple-granularity locking scheme. A more efficient way to gain this knowledge is to introduce a new lock mode, called *Intention lock mode*.

**Intention Mode Lock**

In addition to **S** and **X** lock modes, there are three additional lock modes with multiple granularities:

- **Intention-Shared (IS):** explicit locking at a lower level of the tree but only with shared locks.
- **Intention-Exclusive (IX):** explicit locking at a lower level with exclusive or shared locks.
- **Shared & Intention-Exclusive (SIX):** the subtree rooted by that node is locked explicitly in shared mode and explicit locking is being done at a lower level with exclusive mode locks.

The compatibility matrix for these lock modes are described below:

| | IS | IX | S | SIX | X |
|---|---|---|---|---|---|
| IS | ✔ | ✔ | ✔ | ✔ | ✗ |
| IX | ✔ | ✔ | ✗ | ✗ | ✗ |
| S | ✔ | ✗ | ✔ | ✗ | ✗ |
| SIX | ✔ | ✗ | ✗ | ✗ | ✗ |
| X | ✗ | ✗ | ✗ | ✗ | ✗ |

**IS** : Intention Shared
**IX** : Intention Exclusive
**S** : Shared
**X** : Exclusive
**SIX** : Shared & Intention Exclusive

*Multi Granularity tree Hierarchy*

The multiple-granularity locking protocol uses the intention lock modes to ensure serializability. It requires that a transaction $T_i$ that attempts to lock a node must follow these protocols:

- Transaction $T_i$ must follow the lock-compatibility matrix.
- Transaction $T_i$ must lock the root of the tree first, and it can lock it in any mode.
- Transaction $T_i$ can lock a node in S or IS mode only if $T_i$ currently has the parent of the node-locked in either IX or IS mode.
- Transaction $T_i$ can lock a node in X, SIX, or IX mode only if $T_i$ currently has the parent of the node-locked in either IX or SIX modes.
- Transaction $T_i$ can lock a node only if $T_i$ has not previously unlocked any node (i.e., $T_i$ is two-phase).
- Transaction $T_i$ can unlock a node only if $T_i$ currently has none of the children of the node-locked.

Observe that the multiple-granularity protocol requires that locks be acquired in top-down (root-to-leaf) order, whereas locks must be released in bottom-up (leaf to-root) order.
As an illustration of the protocol, consider the tree given above and the transactions:

- Say transaction $T_1$ reads record $R_{a2}$ in file $F_a$. Then, $T_1$ needs to lock the database, area $A_1$, and $F_a$ in IS mode (and in that order), and finally to lock $R_{a2}$ in S mode.
- Say transaction $T_2$ modifies record $R_{a9}$ in file $F_a$ . Then, $T_2$ needs to lock the database, area $A_1$, and file $F_a$ (and in that order) in IX mode, and at last to lock $R_{a9}$ in X mode.
- Say transaction $T_3$ reads all the records in file $F_a$. Then, $T_3$ needs to lock the database and area $A_1$ (and in that order) in IS mode, and at last to lock $F_a$ in S mode.
- Say transaction $T_4$ reads the entire database. It can do so after locking the database in S mode.

**T**ransactions $T_1$, $T_3$, and $T_4$ can access the database concurrently. Transaction $T_2$ can execute concurrently with $T_1$, but not with either $T_3$ or $T_4$. This protocol enhances concurrency and reduces lock overhead. Deadlock is still possible in the multiple-granularity protocol, as it is in the two-phase locking protocol. These can be eliminated by using certain deadlock elimination techniques.

**Multiple Granularity Locking**

Multiple granularity locking is a locking mechanism that provides different levels of locks for different database objects. It allows for different locks at different levels of granularity. This mechanism allows multiple transactions to lock different levels of granularity, ensuring that conflicts are minimized, and concurrency is maximized.

To illustrate, let's consider a tree structure that has four levels of nodes. The top level represents the entire database, and below it are nodes of type "area", which represent specific areas of the database. Each area has child nodes called "files", and each file represents a specific subset of data within that area. Importantly, no file can span more than one area.

Finally, each file has child nodes called "records", which represent individual units of data within the file. Like files, each record is a child node of its corresponding file and cannot be present in more than one file.

**Multiple granularity locking uses two types of locks**

**i) Shared Lock**

It allows multiple transactions to read the same data simultaneously. It is used to prevent other transactions from modifying the data while a transaction is reading it.

**ii) Exclusive Lock**

It prevents any other transaction from accessing the data. It is used to prevent other transactions from reading or modifying the data while a transaction is writing to it.

**Different Types of Intention Mode Locks in Multiple Granularity**

Intention mode locks are a type of lock used in multiple granularities locking that allows multiple transactions to acquire locks on the same resource, but with different levels of access.

There are three types of intention mode locks in multiple granularity locking

**i) Intent Shared (IS) Locks**

This lock is used when a transaction needs to read a resource but does not intend to modify it. It indicates that the transaction wants to acquire a Shared lock on a resource.

**ii) Intent Exclusive (IX) Locks**

This lock is used when a transaction needs to modify a resource but does not intend to share it. It indicates that the transaction wants to acquire an Exclusive lock on a resource.

**iii) Shared with Intent Exclusive (SIX) Locks**

This lock is used when a transaction intends to acquire both Shared and Exclusive locks on a resource. It indicates that the transaction wants to acquire an Exclusive lock on a resource after acquiring Shared locks on other resources.

These intention mode locks are used to optimize the locking mechanism in a database by allowing transactions to acquire locks on multiple resources in a coordinated manner. They help prevent deadlocks and improve concurrency in a database system.

Intention lock modes are utilized in the multiple-granularity locking protocol to ensure serializability. According to this protocol, when a transaction (T) attempts to lock a node, it must adhere to the following guidelines

- Transaction T must follow the lock-compatibility matrix.
- Transaction T must initially lock the root of the tree in any mode.
- Transaction T may only lock a node in S or IS mode if it has already locked the node's parent in either IS or IX mode.
- Transaction T may only lock a node in IX, SIX, or X mode if it has already locked the parent of the node in either SIX or IX modes.
- Transaction T may only lock a node if it has not yet unlocked any nodes (i.e., it is two-phase).
- Transaction T may only unlock a node if it is not currently holding any locks on its child nodes.

In the multiple granularity protocol, locks must be acquired in a top-down (root-to-leaf) manner, while they must be released in a bottom-up (leaf-to-root) manner. This ensures that transactions can access shared resources efficiently while avoiding deadlocks.

**Advantages of Multiple Granularity Locking**

**Increased Concurrency**

Multiple granularity locking allows for multiple transactions to access different levels of granularity concurrently, thereby increasing concurrency.

**Reduced Locking Overhead**

Multiple granularity locking allows for locks to be set at different levels of granularity, reducing the locking overhead.

**Improved Performance**

Multiple granularity locking provides finer-grained control over locks, improving performance by reducing the number of conflicts and deadlocks.

**Granularity Levels**

**Table-level Locking**

Table-level locking locks an entire table, preventing other transactions from accessing the table while a transaction is using it.

**Page-level Locking**

Page-level locking locks a specific page within a table, preventing other transactions from accessing the page while a transaction is using it.

**Row-level Locking**

Row-level locking locks a specific row within a table, preventing other transactions from accessing the row while a transaction is using it.

**Deadlocks in Multiple Granularity Locking**

A deadlock occurs when two or more transactions are waiting for each other to release their locks. Deadlocks can be detected using a timeout mechanism. Deadlock can be resolved by rolling back one or more transactions involved in the deadlock. Deadlocks can also be prevented by using strict locking protocols.

**Implementation of Multiple Granularity Locking**
Multiple granularity locking can be implemented in DBMS by defining different levels of granularity for the database objects and using locking protocols that support multiple granularity locking. The implementation of multiple granularity locking can be challenging due to the complexity of the locking protocols.

## 3.14 Deadlock Handling:

In database management systems (DBMS) a deadlock occurs when two or more transactions are unable to the proceed because each transaction is waiting for the other to the release locks on resources. This situation creates a cycle of the dependencies where no transaction can continue leading to the standstill in the system. The Deadlocks can severely impact the performance and reliability of a DBMS making it crucial to the understand and manage them effectively.

**What is Deadlock?**
The Deadlock is a condition in a multi-user database environment where transactions are unable to the complete because they are each waiting for the resources held by other transactions. This results in a cycle of the dependencies where no transaction can proceed.

**Characteristics of Deadlock**

- Mutual Exclusion: Only one transaction can hold a particular resource at a time.
- Hold and Wait: The Transactions holding resources may request additional resources held by others.
- No Preemption: The Resources cannot be forcibly taken from the transaction holding them.
- Circular Wait: A cycle of transactions exists where each transaction is waiting for the resource held by the next transaction in the cycle.

In a database management system (DBMS), a deadlock occurs when two or more transactions are waiting for each other to release resources, such as locks on database objects, that they need to complete their operations. As a result, none of the transactions can proceed, leading to a situation where they are stuck or "deadlocked."

Deadlocks can happen in multi-user environments when two or more transactions are running concurrently and try to access the same data in a different order. When this happens, one transaction may hold a lock on a resource that another transaction needs, while the second transaction may hold a lock on a resource that the first transaction needs. Both transactions are then blocked, waiting for the other to release the resource they need.

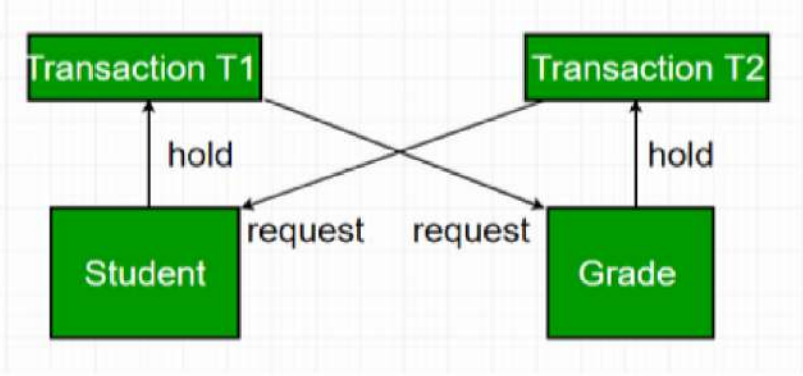
DBMSs often use various techniques to detect and resolve deadlocks automatically. These techniques include timeout mechanisms, where a transaction is forced to release its locks after a certain period of time, and deadlock detection algorithms, which periodically scan the transaction log for deadlock cycles and then choose a transaction to abort to resolve the deadlock.

It is also possible to prevent deadlocks by careful design of transactions, such as always acquiring locks in the same order or releasing locks as soon as possible. Proper design of the database schema and application can also help to minimize the likelihood of deadlocks. In a database, a deadlock is an unwanted situation in which two or more transactions are waiting indefinitely for

one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as it brings the whole system to a Halt.

**Example** – let us understand the concept of deadlock suppose, Transaction T1 holds a lock on some rows in the Students table and **needs to update** some rows in the Grades table. Simultaneously, Transaction **T2 holds** locks on those very rows (Which T1 needs to update) in the Grades table **but needs** to update the rows in the Student table **held by Transaction T1**.

Now, the main problem arises. Transaction T1 will wait for transaction T2 to give up the lock, and similarly, transaction T2 will wait for transaction T1 to give up the lock. As a consequence, All activity comes to a halt and remains at a standstill forever unless the DBMS detects the deadlock and aborts one of the transactions.




*Deadlock in DBMS*

## What is Deadlock Avoidance?

When a database is stuck in a deadlock, It is always better to avoid the deadlock rather than restarting or aborting the database. The deadlock avoidance method is suitable for smaller databases whereas the deadlock prevention method is suitable for larger databases.

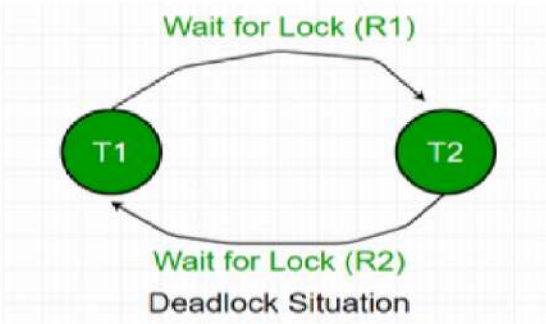
One method of avoiding deadlock is using application-consistent logic. In the above-given example, Transactions that access Students and Grades should always access the tables in the same order. In this way, in the scenario described above, Transaction T1 simply waits for transaction T2 to release the lock on Grades before it begins. When transaction T2 releases the lock, Transaction T1 can proceed freely.

Another method for avoiding deadlock is to apply both the row-level locking mechanism and the READ COMMITTED isolation level. However, It does not guarantee to remove deadlocks completely.

## What is Deadlock Detection?

When a transaction waits indefinitely to obtain a lock, The database management system should detect whether the transaction is involved in a deadlock or not.

**Wait-for-graph** is one of the methods for detecting the deadlock situation. This method is suitable for smaller databases. In this method, a graph is drawn based on the transaction and its lock on the resource. If the graph created has a closed loop or a cycle, then there is a deadlock. For the above-mentioned scenario, the Wait-For graph is drawn below:



## What is Deadlock Prevention?

For a large database, the deadlock prevention method is suitable. A deadlock can be prevented if the resources are allocated in such a way that a deadlock never occurs. The DBMS analyzes the operations whether they can create a deadlock situation or not, If they do, that transaction is never allowed to be executed.

Deadlock prevention mechanism proposes two schemes:

- **Wait-Die Scheme:** In this scheme, If a transaction requests a resource that is locked by another transaction, then the DBMS simply checks the timestamp of both transactions and allows the older transaction to wait until the resource is available for execution.
  Suppose, there are two transactions T1 and T2, and Let the timestamp of any transaction T be TS (T). Now, If there is a lock on T2 by some other transaction and T1 is requesting resources held by T2, then DBMS performs the following actions:
  Checks if TS (T1) < TS (T2) – if T1 is the older transaction and T2 has held some resource, then it allows T1 to wait until resource is available for execution. That means if a younger transaction has locked some resource and an older transaction is waiting for it, then an older transaction is allowed to wait for it till it is available. If T1 is an older transaction and has held some resource with it and if T2 is waiting for it, then T2 is killed and restarted later with random delay but with the same timestamp. i.e. if the older transaction has held some resource and the younger transaction waits for the resource, then the younger transaction is killed and restarted with a very minute delay with the same timestamp.
  This scheme allows the older transaction to wait but kills the younger one.
- **Wound Wait Scheme:** In this scheme, if an older transaction requests for a resource held by a younger transaction, then an older transaction forces a younger transaction to kill the transaction and release the resource. The younger transaction is restarted with a minute delay but with the same timestamp. If the younger transaction is requesting a resource that is held by an older one, then the younger transaction is asked to wait till the older one releases it.

The following table lists the differences between Wait – Die and Wound -Wait scheme prevention schemes:

| Wait – Die | Wound -Wait |
|---|---|
| It is based on a non-preemptive technique. | It is based on a preemptive technique. |
| In this, older transactions must wait for the younger one to release its data items. | In this, older transactions never wait for younger transactions. |
| The number of aborts and rollbacks is higher in these techniques. | In this, the number of aborts and rollback is lesser. |

## Applications

1. **Delayed Transactions:** Deadlocks can cause transactions to be delayed, as the resources they need are being held by other transactions. This can lead to slower response times and longer wait times for users.
2. **Lost Transactions:** In some cases, deadlocks can cause transactions to be lost or aborted, which can result in data inconsistencies or other issues.
3. **Reduced Concurrency:** Deadlocks can reduce the level of concurrency in the system, as transactions are blocked waiting for resources to become available. This can lead to slower transaction processing and reduced overall throughput.
4. **Increased Resource Usage:** Deadlocks can result in increased resource usage, as transactions that are blocked waiting for resources to become available continue to consume system resources. This can lead to performance degradation and increased resource contention.
5. **Reduced User Satisfaction:** Deadlocks can lead to a perception of poor system performance and can reduce user satisfaction with the application. This can have a negative impact on user adoption and retention.

## Features of Deadlock in a DBMS

1. **Mutual Exclusion:** Each resource can be held by only one transaction at a time, and other transactions must wait for it to be released.
2. **Hold and Wait:** Transactions can request resources while holding on to resources already allocated to them.
3. **No Preemption:** Resources cannot be taken away from a transaction forcibly, and the transaction must release them voluntarily.
4. **Circular Wait:** Transactions are waiting for resources in a circular chain, where each transaction is waiting for a resource held by the next transaction in the chain.
5. **Indefinite Blocking:** Transactions are blocked indefinitely, waiting for resources to become available, and no transaction can proceed.
6. **System Stagnation:** Deadlock leads to system stagnation, where no transaction can proceed, and the system is unable to make any progress.
7. **Inconsistent Data:** Deadlock can lead to inconsistent data if transactions are unable to complete and leave the database in an intermediate state.
8. **Difficult to Detect and Resolve:** Deadlock can be difficult to detect and resolve, as it may involve multiple transactions, resources, and dependencies.

## Disadvantages

1. **System downtime:** Deadlock can cause system downtime, which can result in loss of productivity and revenue for businesses that rely on the DBMS.
2. **Resource waste:** When transactions are waiting for resources, these resources are not being used, leading to wasted resources and decreased system efficiency.
3. **Reduced concurrency:** Deadlock can lead to a decrease in system concurrency, which can result in slower transaction processing and reduced throughput.
4. **Complex resolution:** Resolving deadlock can be a complex and time-consuming process, requiring system administrators to intervene and manually resolve the deadlock.
5. **Increased system overhead:** The mechanisms used to detect and resolve deadlock, such as timeouts and rollbacks, can increase system overhead, leading to decreased performance.

## 3.15 Recovery Concepts:

Database Systems like any other computer system, are subject to failures but the data stored in them must be available as and when required. When a database fails it must possess the facilities for fast recovery. It must also have atomicity i.e. either transactions are completed successfully and committed (the effect is recorded permanently in the database) or the transaction should have no effect on the database.

**Types of Recovery Techniques in DBMS**

Database recovery techniques are used in database management systems (DBMS) to restore a database to a consistent state after a failure or error has occurred. The main goal of recovery techniques is to ensure data integrity and consistency and prevent data loss.

There are mainly two types of recovery techniques used in DBMS 

**Rollback/Undo Recovery Technique**

- **Commit/Redo Recovery Technique**
- **Check Point Recovery Technique**

**Rollback/Undo Recovery Technique**

**The rollback/undo recovery technique** is based on the principle of backing out or undoing the effects of a transaction that has not been completed successfully due to a system failure or error. This technique is accomplished by undoing the changes made by the transaction using the log records stored in the transaction log. The transaction log contains a record of all the transactions that have been performed on the database. The system uses the log records to undo the changes made by the failed transaction and restore the database to its previous state.

**Commit/Redo Recovery Technique**

**The commit/redo recovery technique** is based on the principle of reapplying the changes made by a transaction that has been completed successfully to the database. This technique is accomplished by using the log records stored in the transaction log to redo the changes made by the transaction that was in progress at the time of the failure or error. The system uses the log records to reapply the changes made by the transaction and restore the database to its most recent consistent state.

**Checkpoint Recovery Technique**

**Checkpoint Recovery** is a technique used to improve data integrity and system stability, especially in databases and distributed systems. It entails preserving the system's state at regular intervals, known as checkpoints, at which all ongoing transactions are either completed or not initiated. This saved state, which includes memory and CPU registers, is kept in stable, non-volatile storage so that it can withstand system crashes. In the event of a breakdown, the system can be restored to the most recent checkpoint, which reduces data loss and downtime. The frequency of checkpoint formation is carefully regulated to decrease system overhead while ensuring that recent data may be restored quickly.

**Database Systems**

There are both automatic and non-automatic ways for both, backing up data and recovery from any failure situations. The techniques used to recover lost data due to system crashes, transaction errors, viruses, catastrophic failure, incorrect command execution, etc. are database recovery techniques. So to prevent data loss recovery techniques based on deferred updates and immediate updates or backing up data can be used. Recovery techniques are heavily dependent upon the existence of a special file known as a **system log**. It contains information about the start and end of each transaction and any updates which occur during the **transaction**. The log keeps track of all transaction operations that affect the values of database items.

This information is needed to recover from transaction failure.

- **The log is kept on disk start_transaction(T):** This log entry records that transaction T starts the execution.
- **read_item(T, X):** This log entry records that transaction T reads the value of database item X.

**write_item(T, X, old_value, new_value):** This log entry records that transaction T changes the value of the database item X from old_value to new_value. The old value is sometimes known as a before an image of X, and the new value is known as an afterimage of X.

- **commit(T):** This log entry records that transaction T has completed all accesses to the database successfully and its effect can be committed (recorded permanently) to the database.
- **abort(T):** This records that transaction T has been aborted.

- **checkpoint:** A checkpoint is a mechanism where all the previous logs are removed from the system and stored permanently in a storage disk. Checkpoint declares a point before which the DBMS was in a consistent state, and all the transactions were committed.

A transaction T reaches its **commit** point when all its operations that access the database have been executed successfully i.e. the transaction has reached the point at which it will not **abort** (terminate without completing). Once committed, the transaction is permanently recorded in the database. Commitment always involves writing a commit entry to the log and writing the log to disk. At the time of a system crash, the item is searched back in the log for all transactions T that have written a start_transaction(T) entry into the log but have not written a commit(T) entry yet; these transactions may have to be rolled back to undo their effect on the database during the recovery process.

- **Undoing:** If a transaction crashes, then the recovery manager may undo transactions i.e. reverse the operations of a transaction. This involves examining a transaction for the log entry write_item(T, x, old_value, new_value) and setting the value of item x in the database to old-value. There are two major techniques for recovery from non-catastrophic transaction failures: deferred updates and immediate updates.
- **Deferred Update:** This technique does not physically update the database on disk until a transaction has reached its commit point. Before reaching commit, all transaction updates are recorded in the local transaction workspace. If a transaction fails before reaching its commit point, it will not have changed the database in any way so UNDO is not needed. It may be necessary to REDO the effect of the operations that are recorded in the local transaction workspace, because their effect may not yet have been written in the database. Hence, a deferred update is also known as the **No-undo/redo algorithm.**
- **Immediate Update:** In the immediate update, the database may be updated by some operations of a transaction before the transaction reaches its commit point. However, these operations are recorded in a log on disk before they are applied to the database, making recovery still possible. If a transaction fails to reach its commit point, the effect of its operation must be undone i.e. the transaction must be rolled back hence we require both undo and redo. This technique is known as **undo/redo algorithm.**
- **Caching/Buffering:** In this one or more disk pages that include data items to be updated are cached into main memory buffers and then updated in memory before being written back to disk. A collection of in-memory buffers called the DBMS cache is kept under the control of DBMS for holding these buffers. A directory is used to keep track of which database items are in the buffer. A dirty bit is associated with each buffer, which is 0 if the buffer is not modified else 1 if modified.
- **Shadow Paging:** It provides atomicity and durability. A directory with n entries is constructed, where the ith entry points to the ith database page on the link. When a transaction began executing the current directory is copied into a shadow directory. When a page is to be modified, a shadow page is allocated in which changes are made and when it is ready to become durable, all pages that refer to the original are updated to refer new replacement page.
- **Backward Recovery:** The term "**Rollback**" and "**UNDO"** can also refer to backward recovery. When a backup of the data is not available and previous modifications need to be undone, this technique can be helpful. With the backward recovery method, unused modifications are removed and the database is returned to its prior condition. All adjustments

made during the previous traction are reversed during the backward recovery. In other words, it reprocesses valid transactions and undoes the erroneous database updates.

- **Forward Recovery:** "**Roll forward**" and "**REDO"** refers to forwarding recovery. When a database needs to be updated with all changes verified, this forward recovery technique is helpful. Some failed transactions in this database are applied to the database to roll those modifications forward. In other words, the database is restored using preserved data and valid transactions counted by their past saves.

**Backup Techniques**

There are different types of Backup Techniques. Some of them are listed below.

- **Full database Backup:** In this full database including data and database, Meta information needed to restore the whole database, including full-text catalogs are backed up in a predefined time series.
- **Differential Backup:** It stores only the data changes that have occurred since the last full database backup. When some data has changed many times since the last full database backup, a differential backup stores the most recent version of the changed data. For this first, we need to restore a full database backup.
- **Transaction Log Backup:** In this, all events that have occurred in the database, like a record of every single statement executed is backed up. It is the backup of transaction log entries and contains all transactions that had happened to the database. Through this, the database can be recovered to a specific point in time. It is even possible to perform a backup from a transaction log if the data files are destroyed and not even a single committed transaction is lost.

**Recovery from Deadlock**

If the wait for graph which is used for deadlock detection contains a deadlock situation i.e. there exists cycles in it then those cycles should be removed to recover from the deadlock. The most widely used technique of recovering from a deadlock is to rollback one or more transactions till the system no longer displays a deadlock condition.

**The selection of the transactions to be rolled back is based on the following deliberations:**

**Selection of victim:** There may be many transactions which are involved in a deadlock i..e deadlocked transaction. So to recover from the deadlock some of the transaction should be rolled back, out of the possible transactions causing a deadlock. The one that is rolled back is known as victim transaction and the mechanism is known as victim election. The transactions to be rolled back are the one which has just started or has not made many changes. Avoid selecting transactions that have made many updates and have been running for a long time.

**Rollback:** Once the selection of the transaction to be rolled back is decided we should find out how far the current transaction should be rolled back. One of the simplest solution is the total rollback i.e. abort the transaction and restart it. However, the transaction should be rolled back to the extent required to break the deadlock. Also, the additional information of the state of currently executing transactions should be maintained.

**Starvation:** To recover from the deadlock, we must ensure that the same transaction should not be selected again and again as a victim to rollback. The transaction will never complete if the type of

situation is not avoided. To avoid starvation, only a finite number of times a transaction should be picked up as a victim. A widely used solution is to include the number of rollbacks of the transaction that is selected as the victim.

## 3.16 Recovery Based on Deferred:

### Concept of Log

• Log is the most commonly used structure for recording the modifications that is to be made in the actual database. Hence during the recovery procedure a log file is 50 maintained.

• A log record maintains four types of operations. Depending upon the type of operations there are four types of log records -

1. <start> Log record: It is represented as $<T_i$, Start>

2. <Update> Log record

3. <Commit> Log record: It is represented as $<T_i$, Commit>

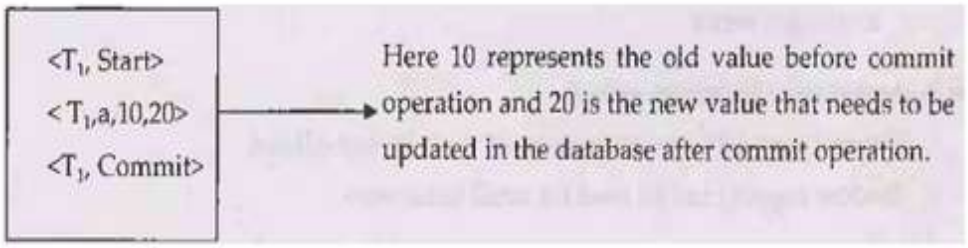
4. <Abort> Log record: It is represented as $<T_i$, Abort>

• The log contains various fields as shown in following Fig. 3.19.1. This structure is for <update> operation

| Transaction ID($T_i$) | Data Item Name | Old Value of Data Item | New Value of Data Item |
|---|---|---|---|

Fig. 3.19.1

• For example: The sample log file is




Here 10 represents the old value before commit operation and 20 is the new value that needs to be updated in the database after commit operation.

• The log must be maintained on the stable storage and the entries in the log file are maintained before actually updating the physical database.

• There are two approaches used for log based recovery technique - Deferred Database Modification and Immediate Database Modification.

## REDO and UNDO Operation

During transaction execution, the updates are recorded only in the log and in the cache buffers. After the transaction reaches its commit point and the log is force written to disk and the updates are recorded in the database.

In order to maintain the atomicity of transaction, the operations can be redone or

undone.

**UNDO:** This is an operation in which we restore all the old values (BFIM - BeFore Modification Image) onto the disk. This is called roll-back operation.

**REDO:**This is an operation in which all the modified values(AFIM - After Modification Image) are restored onto the disk. This is called roll-forward operation.

These operations are recorded in the log as they happen.

**Difference between UNDO and REDO**

| Sr. No. | UNDO | REDO |
|---|---|---|
| 1. | Makes a change go away. | Reproduces a change. |
| 2. | Used for rollback and read consistency. | Used for rolling forward the changes. |
| 3. | Protects the database from inconsistent reads. | Protects from data loss. |

## Write Ahead Logging Rule

• Before a block of data in main memory is output to the database, all log records pertaining to data in that block must have been output to stable storage. This rule is called the Write-Ahead Logging (WAL)

• This rule is necessary because - In the event of a crash or ROLLBACK, the original content contained in the rollback journal is played back into the database file to revert the database file to its original state.

## Deferred Database Modification

• In this technique, the database is not updated immediately.

• Only log file is updated on each transaction.

• When the transaction reaches to its commit point, then only the database is physically updated from the log file.

• In this technique, if a transaction fails before reaching to its commit point, it will not have changed database anyway. Hence there is no need for the UNDO operation. The REDO operation is required to record the operations from log file to physical database. Hence deferred database modification technique is also called as NO UNDO/REDO algorithm.

• **For example:**

**Consider two transactions $T_1$ and $T_2$ as follows:**

| $T_1$ | $T_2$ |
|---|---|
| Read (A, a) | Read (C, c) |
| a = a – 10 | c = c – 20 |
| Write (A, a) | Write (C, c) |
| Read (B, b) | |
| b = b + 10 | |
| Write (B, b) | |

If $T_1$ and $T_2$ are executed serially with initial values of A = 100, B = 200 and C = 300, then the state of log and database if crash occurs

a) Just after write (B, b)

b) Just after write (C, c)

c) Just after <$T_2$, commit>

The result of above 3 scenarios is as follows:

Initially the log and database will be

| Log | Database |
|---|---|
| <$T_1$, Start> | |
| <$T_1$, A, 90> | |
| <$T_1$, B, 210> | |
| <$T_1$, Commit> | |
| | A = 90 |
| | B = 210 |
| <$T_2$, Start> | |
| <$T_2$, C, 280> | |
| <$T_2$, Commit> | |
| | C = 280 |

**a) Just after write (B, b)**

Just after write operation, no commit record appears in log. Hence no write operation is performed on database. So database retains only old values. Hence A = 100 and B = 200 respectively.

Thus the system comes back to original position and no redo operation take place. The incomplete transaction of $T_1$ can be deleted from log.

**b) Just after write (C, c)**

The state of log records is as follows

Note that crash occurs before $T_2$ commits. At this point $T_1$ is completed successfully, so new values of A and B are written from log to database. But as $T_2$ is not committed, there is no redo ($T_2$) and the incomplete transaction $T_2$ can be deleted from log.

The redo ($T_1$) is done as < $T_1$, commit> gets executed. Therefore A = 90, B = 210 and C=300 are the values for database.

**c) Just after < T2, commit>**

The log records are as follows:

<$T_1$, Start>

<$T_1$, A, 90>

<$T_1$, B, 210>

<$T_1$, Commit>

<$T_2$, Start>

<T2, 6, 280>

<$T_2$, Commit>

← Crash occurs here

Clearly both $T_1$ and $T_2$ reached at commit point and then crash occurs. So both redo ($T_1$) and redo ($T_2$) are done and updated values will be A = 90, B = 210, C = 280.

## Log-Based Recovery

- The log is a sequence of records. Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there.
- If any operation is performed on the database, then it will be recorded in the log.
- But the process of storing the logs should be done before the actual transaction is applied in the database.

Let's assume there is a transaction to modify the City of a student. The following logs are written for this transaction.
When the transaction is initiated, then it writes 'start' log.

<Tn, Start>

When the transaction modifies the City from 'Noida' to 'Bangalore', then another log is written to the file.

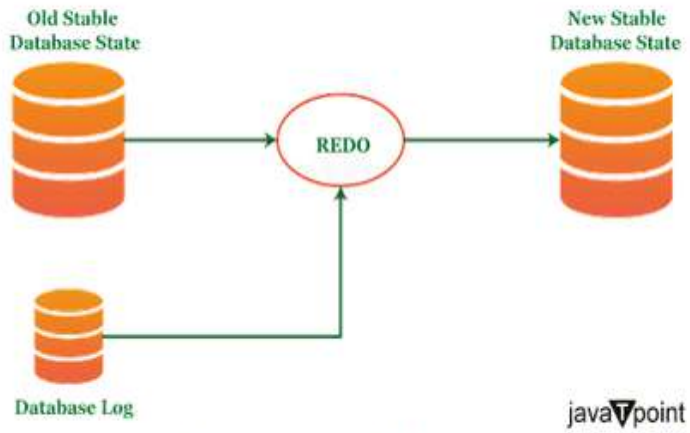
<Tn, City, 'Noida', 'Bangalore' >

When the transaction is finished, then it writes another log to indicate the end of the transaction.

<Tn, Commit>

### I. Deferred database modification:

- The deferred modification technique occurs if the transaction does not modify the database until it has committed.
- In this method, all the logs are created and stored in the stable storage, and the database is updated when a transaction commits.




**Recovery technique based on deferred update in a single user environment:**

A recovery in a single-user environment is much easier than an environment in which multiple users are running transactions simultaneously. To recover using the deferred update process for all committed and initiated transactions before the failure, follow these steps:

- In this, search forward through the log for the most recent checkpoint record. This is the last time the transaction data was sent to disk. All transactions that were started and committed before the last checkpoint do not need to do anything because the data has already been saved.
- For any transaction T, for which the log contains both a [Start_transaction, T] record and a [Commit, T] records that appears in the log after the last checkpoint. Database management uses the log records to redo the transaction and update the database using new value in the transaction log.
- For any transaction T, in which the log only has a [Start_transaction, T] record but not a [Commit, T] record since the last checkpoint, nothing needs to be done because the database has never been updated.

To illustrate the concept of deferred update in a single user environment, consider an example involving two transactions T1 and T2. Transaction T1 transfers Rs. 200 from account A to account B and transaction T2 withdraws Rs. 100 from account C. The initial balances of accounts A, B and C are Rs. 1000, Rs. 1500 and Rs. 2000 respectively. Suppose transactions T1 and T2 are executed sequentially with T1 followed by T2.

| T1 | T2 |
|---|---|
| Read (A) | Read I |
| A : = A - 200 | C : = C - 100 |
| Write (A) | Write I |
| Read (B) | |
| B := B + 200 | |
| Write (B) | |

**The order in which these transactions will be stored in system log will be discussed below:**

- When a transaction starts a transaction start record is written in the system log, so here when transaction starts T1, a start record is written on the log. Similarly, when a transaction T2 starts a start record is written in the system log.
- When a write operation is performed, a write log record is written which contains all the data given earlier. So when transaction T1 performs a Write (A) operation, a write record.

[Write, T1, A, 800] is written on the system log. One should note at this point, updates are not written to disk. Similarly on execution of Write (B) operation by transaction T1 and Write(C) operation by transaction T2, appropriate write log records are written.

- When the transaction is committed, a commit record is written to the system log. So if transaction T1 is about to be committed, a transaction commit record is written to the system log.

[Commit, T1]

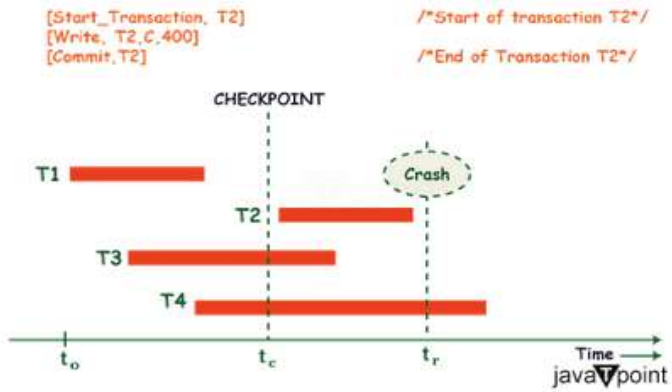
All log records for the disk and transaction is committed.Similarly for transaction T2 is about to be committed, a transaction commit record is written to the system log.

The full contents of the system log when these two transactions (T1 and T2) are executed sequence T1 and T2 are shown below:

| Log | Database |
|---|---|
| [Start_transaction, T1] | |
| [Write , T1 , A, 800] | |
| [Write , T1 , B, 1700] | |
| [Commit, T1] | A = 800 |
| | B = 1700 |
| [Start_transaction, T2] | |
| [Write , T2 , C, 1900] | |
| [Commit , T2] | |
| | C = 1900 |

Let's take an example where a system crashes before a transaction completes. So the system handles failure that result is loss of information using the log. In case of failure, the log files are examined starting with the last entry till the most recent checkpoint. All transaction start and transaction commit records before the abort record should be redone. The redo method performs all the writes to the database in the order they were written to the log. If this updation has already been performed then the write operation has no effect on the data item when it is redone. This guarantees that all data items will be updated properly. All those transactions that only contains start record but no commit record need to be restarted.

**The figure below shows different situations when a system crash occurs:**




The redo operation must be mandatory in a deferred update, meaning running it repeatedly is the same as running it only once. If the system fails during the recovery process, a succeeding recovery attempts may redo some of the write operations that were already redone during the first recovery process, as if the recovery operations were restarted from the beginning. Because the redo write value is currently in the database, the result of a successful second recovery process on redo is the same as if the redo had succeeded the first time.

| Condition 1 | Condition 2 | Condition 3 |
|---|---|---|
| [Start_transaction, T] | [Start_transaction, T1] | [Start_transaction, T1] |
| [Write , T1 , A, 800] | [Write , T1 , A, 800] | [Write , T1 , A, 800] |
| [Write , T1 , B, 1700] | [Write , T1 , B, 1700] | [Write , T1 , B, 1700] |
| [abort, T1] | [Commit, T1] | [Commit, T1] |
| | [Start_transaction, T2] | [Start_transaction, T2] |
| crash | [Write , T2 , C, 1900] | [Write , T2 , C, 1900] |
| | [abort, T1] | [Commit , T2] |
| | | [abort, T1] |
| | crash | |
| | | crash |
| Whenever recovery process starts no redo action needs to be taken because no commit record appears in the log and value of Account A and B remains same i.e. 1000 and 1500 respectively. | Whenever recovery process starts it finds that transaction T1 is already committed in the log. So operation Redo for transaction T1 must be performed and all data items updated by transaction T1 are copied into database. A = 800 and B = 1700. But the transaction T2 is not yet committed in the log so there is no need to redo the transaction T2. This transaction T2 is deleted from log and re-executed so value of data item C = 2000. | Whenever the recovery process starts it finds that there are committed records in the log for both transactions T1 and T2. This results in redo operations being performed for both T1 and T2. Hence the value of data items A, B and C is Rs. 800, Rs. 1700 and Rs. 1900 respectively. |

| No Redo | Redo (T1)<br><br>T2 must be re-executed | Redo (T1) , Redo (T1) |
|---|---|---|
| A = 1000, B = 1500, C = 2000 | A = 800, B = 1700, C = 2000 | A = 800, B =1700, C = 1900 |